Is a Dependency-Change Block in a Context File Worth Its Tokens? A Break-Even Model and Coding-Agent Studies

Mohammad Hijjawi
Technical report, October 2026
https://github.com/MohammadHijjawi97/since-cutoff

Abstract

Coding agents know library APIs as of their training cutoff, yet 53% of random popular Python repositories with pins lock a dependency released after it (in 12%, code matches a changed API). Is an always-loaded context-file block listing such changes worth its tokens? `since-cutoff` writes one from a lockfile with no model call: a header naming changed packages, then change lines. We compared it with no block, the header alone, irrelevant content and a generic warning in Claude Code. Opus 5.5 solved its own small tasks almost always unaided, so the block could only change cost; it met two of ten pre-committed efficiency rules (main-study cost 0.80, new-task tokens 0.79; uncorrected across rules), and a replication missed. No pre-committed test distinguished it from the header alone or the generic warning, and other models showed no detectable saving. In one developer's agent contexts, a real-size block adds about 0.4% to a context's cost, but the contexts that need a changed API repay it on their own only once they make up 14–49% of all contexts; an estimated 0.8–7% did. Whether the block pays overall therefore turns on its effect on the other contexts, which our control tasks put at a 6.6% saving but cannot tell from zero. Leaving such blocks off by default is a loss-averse choice, not a finding that they lose money; a live A/B test of about 2,500–2,900 sessions per arm would settle it for a given fleet, and offline tests should include header-only and generic arms in the same batches.

1 Introduction

Every language model has a training cutoff, but the libraries a project depends on keep releasing. When a library changes its public API after the cutoff, a coding agent may write the old form (a removed parameter, a moved import path, a dependency the new release no longer installs) or spend turns reading installed sources to find out what changed. Such exposure is common but mostly latent: of randomly sampled popular Python repositories with pins, 53% lock a direct dependency released after June 2026, the cutoff of the 5.5 models we test, but our matcher finds a changed-API use in only 12%, and an AI judge found none of 48 audited uses broken (§6). A context-file block is paid for in every agent context (a session, or a subagent run that loads the file), so whether to enable one depends on two things: the share f of contexts that need a changed API, and the block's effect on all other contexts.

Benchmarks show that models write outdated or version-incompatible code (e.g., Wang et al., 2025; Misra et al., 2026; Cipollone et al., 2026). Documentation servers such as Context7 (Upstash, 2026) let the agent look up documentation on demand; context files such as AGENTS.md (Agentic AI Foundation, 2026) add guidance to every session, but generic ones rarely improve success and can raise cost by over 20% (Gloaguen et al., 2026).

We ask when such a block is worth its tokens. To answer, we built an open-source tool: `since-cutoff` runs outside the agent loop, and its `sync` command writes a block into AGENTS.md or CLAUDE.md with no model call (Figure 1). Its header names each imported package whose API changed since the model's cutoff, with the pinned and cutoff-era versions; up to five evidence-tagged change lines per package follow. We ran Claude Code with it (Notes) and without it (None) on small Python repositories whose tasks need a changed API, graded by hidden tests, with header-only, irrelevant and generic-warning controls. Both practitioner baselines failed on Claude: Context7 ran into a quota error, and a lockfile-only version list was registered but not run because the Claude CLI login had expired (Appendices D, E).

Opus 5.5 passed 92–94% of original post-cutoff runs and all new-task runs without help, so the block could mainly change cost: it met its pre-committed rule in the main study (cost 20% lower)

and on new tasks (tokens 21% lower, although with Holm correction across that study's three contrasts $p = 0.094$, so our stricter detection convention does not count it), but not in a replication (cost 15% lower, CI includes 1). In the author's own agent contexts, which average 45 model calls, a real-size block adds about 0.4% to a context's cost. If the saving carries over to real repositories and the other contexts cost the same, the block pays once f exceeds about 14–49% of agent contexts; we estimate f at 0.8% (a lower bound) to 7.0%, so the affected contexts alone do not repay it. The answer then depends on the block's effect u on the other contexts: our control tasks put it at a 6.6% saving, which would make the block pay, but cannot tell it from zero (4% dearer to 16% cheaper), and they are short sessions on tasks the model under test wrote. We therefore leave such blocks off by default as a loss-averse choice, and size the live A/B test that would decide (Lesson 2). All tasks were written by Opus 5.5, the only model on which we detect a saving, and no human has checked them; this is the main threat to validity, and our claims hold only for such tasks (§4). We contribute (i) pre-committed studies with these controls, in which no pre-committed test separated the full block from the header alone (Header) on Opus 5.5 (§4–5); (ii) a break-even model in f and the effect u on other contexts, in one unit (model calls), with real block sizes, a measured f and a measured context cost, and an exposure scan (§5–6); (iii) lessons on concurrent controls, drift and live A/B sizing, with a checklist (§7–8); and (iv) the open tool and every data file and script behind this report (§3, Appendix J).

2 Related Work

Version-aware code generation. Benchmarks measure how often models use deprecated, version-specific or conflicting APIs, or fail at migrations (Wang et al., 2025; Wu et al., 2024; Kuhar et al., 2025; Islah et al., 2024; Misra et al., 2026; Ashik et al., 2026; Cipollone et al., 2026; Cheng et al., 2025), and other work updates API knowledge in the weights (Liu et al., 2024; Lin et al., 2026; Liu et al., 2026a; Gu et al., 2026; Tran et al., 2026; Wu et al., 2026). DEPBENCH (Luo et al., 2026) tests coding agents on dependency upgrades with hidden breakage, and SWE-Chain (Lam et al., 2026) has agents implement successive releases. After an upgrade breaks code, DepRepair (Yang et al., 2026) and BBCFixer (Chen et al., 2026) give a model upstream evidence or a filtered diff.

Context files and documentation. On SWE-bench-style tasks (Jimenez et al., 2024), repository-level context files rarely improve success (Gloaguen et al., 2026); others report lower runtime (Lulla et al., 2026) or no correctness effect (Khatri, 2026). In Claude Code, random rule files helped as much as curated ones, which Zhang et al. (2026) attribute to priming that does not depend on content. Agents follow an instruction file even when it contradicts the code (Xu and Wu, 2026); irrelevant rules lower instruction compliance (Liu et al., 2026b), and skills for the wrong version can lower success (Han et al., 2026). Practitioner guides already advise listing dependency versions in context files (Nigh, 2025), a baseline we pre-committed but have not reported (Appendix E), and other work studies what these files contain (Chatlatanagulchai et al., 2025; Polandra et al., 2026). Retrieval can ground code in documentation (Majdoub et al., 2026; Le-Anh et al., 2026; Chen et al., 2025; Jain et al., 2024); Context7 serves it to agents over MCP (Upstash, 2026; Model Context Protocol, 2026).

What is new here. Correcting outdated APIs at prompt time is not new: APILOT (Bai et al., 2025), InsertPrompt (Wang et al., 2025) and an industrial upgrade agent (Tawosi et al., 2025) add lists of outdated APIs, replacement hints or migration documentation to generation or repair. Breaking-change detectors such as `griffe check` (Mazzucotelli, 2026) and AexPy (Du and Ma, 2022) compare two releases for library maintainers. We combine a model-free block built from the lockfile and the model's cutoff, placed in an always-loaded context file, with cost measured in an agent loop against header-only, irrelevant and generic controls.

3 System

`since-cutoff`'s `sync` calls no model, so its block is deterministic given the lockfile, cutoff and registry state, and each note states its evidence.

Pipeline. (1) *Versions.* Exact versions come from the lockfile (uv, Poetry, PDM, pylock, Pipfile), a local virtual environment or pinned requirements. No project code runs. (2) *Cutoff.* The model's training cutoff comes from a public registry (models.dev) or a flag. The *comparison release* is the last stable release uploaded on or before that date.

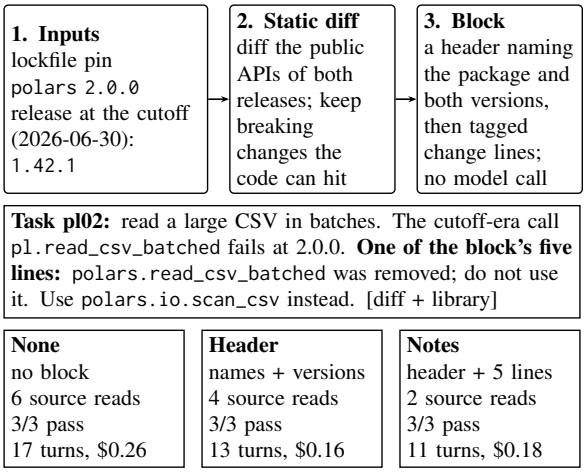


Figure 1: One illustrative task, not a typical effect: `since-cutoff` on pl02 (new-task study; its block named the change, yet Header cost least), one of five change lines (tags: Table 6); medians of three Opus 5.5 runs per arm at list price. Full block: Appendix C.

(3) *API diff.* Both releases are downloaded once and loaded statically with griffe (Mazzucotelli, 2026). Only changes that break code written for the old release are kept: removed or moved objects, removed parameters, parameters that became required, keyword-only or positional-only, changed object kinds, and new deprecations (PEP 702 decorators and warning shims). (4) *Use matching.* A static matcher follows imports and re-exports, checks each call's arguments and labels each use *old form* (written for the old release) or still working. (5) *Block.* A *header* says that the listed packages changed after the cutoff and tells the agent to follow its lines over what it remembers, then names each imported package with breaking changes and both versions ("**polars 2.0.0** (1.42.1 at the cutoff)"). Each package gets up to five *change lines*, each tagged with its evidence (Table 6): the static diff alone, a library deprecation message naming a replacement present in the pinned version, or an object at a new path that keeps the old public names. A replacement is never inferred from similar names alone (a parameter's successor at the same position and type is labelled a probable rename; Table 6), and lines on APIs the code uses come first (Appendix C).

Delivery. `sync` writes the block between markers in AGENTS.md or CLAUDE.md and leaves all other text unchanged. A meta line records the model, cutoff and hashes of the dependencies and block, so `sync --check` and an offline `status` command (both also pre-commit hooks) flag a stale or hand-edited block; `unapply` removes it.

Evaluated configuration. All agent studies but one small follow-up (Appendix E) used release 0.4.1 with a non-default scope (`sync --scope imported --model <id> --target CLAUDE.md`), which writes lines on every changed API of each imported package, called or not, at the default cap of five lines per package. The default `used` scope (changed APIs the code uses) is unevaluated; in our random sample (§6) it writes a block for 13% of repositories (imported scope: 47%). Header-only output, which a deployment would need to run the header alone, is in no release of the tool (the latest release when this report was written is 0.5.0).

Detector quality. Of 154 documented changes in seven libraries, 0.4.1 found 49% [41, 57], AexPy 53% and `griffe check` 55% (`griffe check` found 10 that 0.4.1 missed, none the reverse); all 56 sampled 0.4.1 reports were accurate, and a five-line block holds 8.4% of the 154 (AI-judged; Appendix G); we contribute delivery and evaluation, not a better diff.

Release. `since-cutoff` is released under the MIT licence at `https://github.com/MohammadHijjawi97/since-cutoff`; the main study's tasks, hidden tests and transcripts are public at `https://github.com/MohammadHijjawi97/since-cutoff-benchmark`, and the data and scripts behind this report are in the tool's repository, in `research/dependency-block-study` (Appendix J). The tool has no telemetry, so we have no deployment data.

4 Evaluation Setup

Pre-commitment. Each study's protocol (tasks, hidden-test hashes, frozen notes, analysis code, decision rules) was committed and tagged in git before its first session; we call such a study *pre-committed* (internally, in git; not an external registration). The tags are unsigned and their times are set by us, so they record the order of steps but are no proof of it (Appendix J); several follow-up studies were added after AI-assisted internal reviews of a draft.

Tasks. We use 35 Python tasks, each a small repository with an instruction and hidden tests (Appendix A). The original 24, frozen before the pilot, are seven controls (older changes, no change, an older pin) and 17 *post-cutoff* tasks, whose solution

study	tasks ×runs	arms (sessions)	pass (%) A/C	Notes/None [95% CI] list cost	Notes/None [95% CI] total tokens	p, raw; Holm[d]	H/A cost; tokens	pre-committed rule met?
Opus 5.5: main study primary; replication and new tasks confirmatory; concurrent study for the controls								
Main	17×3	A,C (144)	94.1/100	**0.80** [0.70, 0.90]	0.73 [0.59, 0.87]	<0.001; 0.0011	–	yes (cost[a])
Replication	17×3	A,C,H (153)	92.2/100	0.85 [0.69, 1.01]	0.83 [0.68, 0.99]	0.13; 0.38	0.99; 0.95	no (tok.+cost)
New tasks	8×3	A,C,H (99)	100/95.8	0.83 [0.77, 0.91]	**0.79** [0.68, 0.92]	0.031; 0.094	0.94; 0.86	yes (tokens)
Concurrent[b]	8×3	A,C,H,R,G (165)	100/100	0.79 [0.70, 0.90]	0.75 [0.62, 0.90]	0.13; 0.25	0.80; 0.73	no (family[b])
Smaller and open-weight models: no per-session saving detected								
Sonnet 5.5	17×3	A,C,H (216)	74.5/90.2	0.94 [0.76, 1.13]	0.93 [0.75, 1.14]	0.55; 0.55	1.13; 1.13	no (tok., pass)
Haiku 5.5	17×3	A,C (144)	80.4/86.3	0.99 [0.86, 1.19]	0.92 [0.82, 1.05]	0.23; 0.23	–	no (tok., pass)
Qwen3-Coder[c]	31×3	A,C,H (261)	19.4/24.7	n/a	1.08 [0.87, 1.32]	0.50; 0.50	n/a; 0.79	no (pass, tok.)
Appendices: Context7 arms (216 sessions; quota error) and four follow-ups (209 sessions), one on an older-cutoff model.								

Table 1: Study map: all eleven completed studies (1,607 sessions), results on post-cutoff tasks (tasks × runs per arm; sessions include any control tasks). Arms (Table 2): A None, C Notes, H Header, R Irrelevant, G Generic. Ratios (below 1: cheaper than None): geometric mean over tasks of the ratio of the two arms' median runs (each model's list prices); task-cluster bootstrap CIs. **Bold**: the metric of a pre-committed rule that was met (all ten rules: Table 12); tok.: total tokens. Main: A and C only (360 sessions with the Context7 arms). [a]One of four metrics in the pre-committed rule; its CI stays below 1 after Bonferroni correction. [b]All five arms in the same randomised batches; the pre-committed family was C/G, C/R and G/H (p column: C/G tokens), and C/A and H/A were not in it. [c]Qwen3-Coder-30B-A3B in mini-swe-agent, April 2025 cutoff; a quarter of sessions hit an adapter fault; H/A on the 17 original tasks, not randomised. [d]Exact two-sided sign-flip test on per-task log ratios of the pre-committed metric (cost for the main study and the replication, whose rule needed both cost and tokens; otherwise C/A tokens), raw and Holm-adjusted within the study's family (main: the rule's four metrics; replication, new tasks and Sonnet 5.5: C/A, H/A and C/H; concurrent: C/G, C/R and G/H; Haiku 5.5 and Qwen3-Coder: C/A only). The rules were scored on CIs as written, with no correction across the ten rules; with 8 tasks the smallest raw p is 0.0078, and the smallest Holm p over three contrasts is 0.023.

needs an API changed after the June 2026 cutoff. All 17 are in LLM and agent SDKs (task IDs ant: anthropic, oai: openai, hf: huggingface-hub, mcp). The 11 new tasks, written after the first results, add eight post-cutoff tasks in polars (pl), streamlit (st) and mistralai (mi) and three controls. Claude Code sessions running Opus 5.5, the model under test, wrote all tasks, reference solutions and hidden tests under the author's direction, and a separate session reviewed each set; no human has checked any of the tasks. This is our main threat to validity: such tasks may suit their writer, which may explain its ceiling and a saving seen only on Opus, so every result below is scoped to them. A task was admitted only if it passed four gates: (i) the unmodified repository fails the hidden tests, (ii) a reference solution passes, and a *cutoff-era* solution (iii) fails at the pinned version but (iv) passes at the cutoff-era release. A leak check keeps answer identifiers out of the repository and the instruction.

Agent and arms. We ran Claude Code 2.1.283 (Anthropic, 2026) non-interactively with effort "medium", at most 40 turns and 900 s per session, no network tools, a fresh environment and fixed pins. All arms share a base CLAUDE.md and differ only in the added block (Table 2). The studies ran from 28 September to 23 October 2026 with three runs per task and arm; within each study, the arms ran interleaved in one sitting, in randomised batches of one task and repetition. Table 1 maps all eleven completed studies; all but the case study pre-committed an efficiency rule, and all 1,607 analysed sessions were valid, that is, graded without infrastructure error (two after a re-run); open-weight runs stopped by our adapter's fault count as failures. Exceptions: in two small follow-ups and in the open-weight study's Header arm, the added arm ran hours to a day after the others.

arm	block added to base CLAUDE.md (est. tokens)
A None	none
C Notes	header + lines (285–505; mean 423*)
H Header	Notes' header: packages, pinned and cutoff versions, "follow these lines" (about 244)
R Irrelevant	Header + true change lines of an unused package, length-matched to Notes
G Generic	APIs may be out of date, check installed signatures; no package named (about 218)

Table 2: Arms (letters as in tables and appendices). *Over the 20 non-empty original blocks.

Metrics and analysis. A run passes if every hidden test passes and the pins are unchanged. The unit of analysis is the task. For list-price cost, total tokens and turns, we take each arm's median run per task, form the ratio between arms, and report its geometric mean over tasks with task-cluster bootstrap CIs. The main study's pre-committed rule was met if any of four efficiency metrics improved (failing test runs, turns, wall time, cost; none designated primary); we report cost, whose interval excludes 1 even after Bonferroni correction. Later studies pre-committed total tokens, because the CLI mispriced the smaller models; on Opus, cache reads are about 90% of tokens but 15–16% of cost, so where token and cost ratios disagree we report both. A pre-committed *ceiling rule* forbids pass-rate claims when None passes at least 90% of post-cutoff tasks. One convention holds throughout: a difference is *detected* only if its bootstrap CI excludes 1 (0 for pass rates) and the exact sign-flip test on per-task log ratios, Holm-corrected within the study's pre-committed family, gives $p < 0.05$; otherwise we claim neither a difference nor its absence. Pre-committed rules are scored as written, on CIs (Table 12), so a rule can be met while its difference is not detected; Table 1 gives the raw and Holm-adjusted p of each rule's metric. Control non-inferiority uses exact Clopper-Pearson bounds (Appendix D). Two tasks (ant01, mcp01) were seen in the pilot, and the tool was changed after ant01's pilot runs, so we check every claim without them.

5 Results

Concurrent evidence first. Because None's token use drifted between sittings (Lesson 1), every Opus contrast in Table 1 compares arms run in the same batches. One study ran all five arms together; relative to None, tokens were 0.73 with Header, 0.75 with Notes, 0.81 with Irrelevant and 0.94 with Generic (Q2). Two of the ten pre-committed efficiency rules were met and still hold, both on Opus 5.5 (Table 12); a third, from an earlier Irrelevant study, was met but withdrawn (Lesson 1). Per-task effects of the main study and its replication did not correlate ($r = -0.19$ over 17 tasks), which fits either no stable per-task effect or low per-task reliability (each task's ratio compares medians of three runs); either way the saving is a modest average.

Q1. Does the block lower cost? For Opus 5.5 it does, modestly and on average. In the main study, Notes cost 0.80 of None [0.70, 0.90] ($0.34 against $0.42 per post-cutoff session; raw $p < 0.001$, Holm $p = 0.0011$). On the eight new tasks it used 0.79 of None's tokens, the pre-committed metric: the CI rule was met, but the Holm-adjusted p over the study's three contrasts is 0.094, so by our convention the token saving is not detected; its cost, a secondary metric, was 0.83 of None's (Holm $p = 0.023$ over the same three contrasts in cost). The replication, 24 days later, saved 15% (CI includes 1), mostly on the five tasks using the evaluated vendor's SDK (Appendix D).

Break-even model (exploratory). Three quantities decide whether a block pays: its cost c per agent context, the gross saving s in a context that needs a changed API, and its fractional effect u on the cost K of any other context ($u < 0$: dearer). With a share f of such contexts the net saving per context is

$$N = fs + (1 - f)\,uK - c, \qquad f^* = \frac{c - uK}{s - uK},$$

so the block pays once $f > f^*$ (for $s > uK$). Every length is counted in model calls (distinct assistant messages): $c = b\,(t\,p_r + p_w)$ for b tokens over t model calls at cache-read and cache-write prices p_r, p_w (one cache write). s = $0.087 is the main study's saving plus its block's own cost; we assume that s and u depend neither on b nor on t.

Inputs. f: in the author's 1,308 agent contexts (Appendix I; 1,281 were subagent runs, most of which load CLAUDE.md (Anthropic, 2026)), f_{strict}, the share whose edited code uses a changed API by our matcher, is 0.8% (a lower bound), and f_{upper}, the share that edit a file importing a changed package, is 7.0% (11% if missing imports are imputed, with too narrow an interval because the imputed rates are held fixed). c: the contexts average 45.3 model calls, so at Opus 5.5 list prices (p_r = $0.20, p_w = $8 per million tokens) the median non-empty block of our random sample (704 tokens) costs $0.012 per context; a benchmark session (11.3 calls) would pay $0.007. K: the contexts' logged usage, priced the same way, gives $2.96 per context, so the block adds only c/K = 0.4% to a context's cost.

The cost is small and bounded, but affected contexts alone do not repay it. At $u = 0$ the block pays only once f exceeds 14% (14–49% over real block sizes and savings; 20% among Python-editing contexts, where f is 2.1–17.7%), so at the measured f

its net saving per 1,000 contexts is $-\$11\,[-15, -9]$ at f_{strict} and $-\$6\,[-9, -1]$ at f_{upper} (joint bootstrap of s, f, t and K; Table 13). Only the imputed Python-editing share (28.7%) exceeds its threshold, with an interval that is too narrow. These thresholds also understate the f needed: s was measured where the cutoff-era code fails, while most real changed-API uses we audited still work (§6).

So u decides. Because c/K is so small, a u above 0.2–0.4% would make the block pay at the measured f over all contexts (Figure 4). On the ten pooled control tasks Notes cost 0.93 [0.84, 1.04] of None, that is $u = 6.6\%$; carried to the contexts, it gives $N = +\$184\,[-117, 590]$ per 1,000 contexts at f_{strict} (+\$177 at f_{upper}), positive in 89% of bootstrap draws. That estimate is weak: its interval includes zero, four identical-context controls gave 0.95 (a 5% "effect" from noise alone), and u comes from short (8.9-call) control sessions on tasks the model under test wrote, applied as a constant relative effect to \$3 contexts, 57% of which edit nothing. The data support neither "the block pays" nor "the block loses".

Decision. "Off by default" is therefore a loss-averse choice: it gives up an uncertain gain to avoid a small, bounded loss (\$6–11 per 1,000 contexts at $u = 0$, more if $u < 0$); it is not a finding that the block loses money. A live A/B test of the net effect over all sessions (Lesson 2; about 2,500–2,900 sessions per arm at our spread) settles the question for a given fleet.

Q2. Which part of the block carries the saving? Not detectably the change lines. In the concurrent study the header alone saved about as many tokens as the full block (0.73 against 0.75 of None), and neither was detectably better than Generic (Notes/Generic tokens 0.80 [0.66, 0.98], Holm p = 0.25), though Generic itself saved no detectable tokens. Across studies Header gave 9–97% of the full block's cost saving (29–114% of its token saving), but its own saving was detected in neither 25-task pool, and no pre-committed cost test separated the two (two post hoc tests favoured the full block; Appendix D). One post hoc pattern is worth testing: on the 7 of 25 tasks whose block named the task's change, Notes/Header tokens were 0.66, against 1.03 on the other 18, but the gap rests largely on ant01, changed after the pilot, and is not detected without it ($p = 0.051$). Change lines may save where they name the change; low coverage limited them. A pre-committed lockfile-only version list (arm L), which would test whether naming the pinned packages is enough, was registered but not run (the Claude CLI login had expired; Appendix E).

Q3. Where did we detect a saving? Only on Opus 5.5 in Claude Code. No per-session saving was detected on Sonnet 5.5, Haiku 5.5 or one open-weight agent, Qwen3-Coder (Table 1), nor on an older-cutoff model, whose passes fell from 82% to 71% (CI includes 0; Appendix E); the Sonnet and Haiku studies had 80% power only for token ratios below about 0.74 and 0.83 (Opus main observed 0.73), so their null results say little about whether the saving is specific to Opus. Sonnet 5.5, below ceiling, gave the clearest correctness signal: Notes and Header each passed 15.7 points more post-cutoff runs (Holm $p = 0.19$; Notes +6.7 without pilot tasks), and Notes' cost per solved task was 0.67 [0.47, 0.89] of None's (post hoc and descriptive); an effect on correctness, the block's purpose, is possible but not established. A second open-model study was started and stopped before completion; its partial runs are not analysed (Appendix F). On the 5.5 models, Notes passed all 45 non-empty-block control runs (exact lower bound 93.6%; with nine runs per study, no single study excludes a 10-point drop), and a block lowered post-cutoff success only on hf02 (Haiku 5.5 lost 2 of 3 passes, Sonnet 5.5 one; Appendix C) and mi01.

6 Real-World Exposure

With `since-cutoff` 0.5.0 (static analysis only), we drew 100 random repositories with pinned dependencies and 100 AI-stack ones (directly importing an LLM, agent or model library) from the 15,424 public Python repositories with at least 200 stars pushed in the 90 days before 23 October 2026 (Appendix H). In the random stratum, 53% lock a direct dependency released after the June 2026 cutoff (owner-clustered 95% CI 43–63%), 44% import a package whose API broke between its cutoff-era and pinned releases, and the matcher finds a changed-API use in 12% (6–19%); in the AI-stack stratum, 93%, 91% and 40%. These rates describe existing code, not what an agent will write, so they do not estimate f. The imported-scope block is non-empty in 47% and 96%, with medians of 704 and 1,025 tokens (benchmark mean: 423). An AI judge audited 60 random matches from 24 repositories, without human review: 48 were true uses

step	what to do (evidence)
Exposure	Run `sync --dry-run --scope imported --model <id>`; no block, no need (47% of random repositories get one).
Default	Off, as a loss-averse choice, unless a live A/B test shows a net saving (Lesson 2). Saving only on Opus 5.5, on tasks it wrote; a 704-token block adds about 0.4% to a context's cost and pays at the author's measured f once u exceeds 0.2–0.4%, which our controls neither show nor exclude.
Test	Pass rate first if below ceiling; then cost, against header-only and generic-warning arms in the same batches (Generic is a control; header-only output is in no release); pre-selected affected sessions give s, others u.
Measure	f (the author: 0.8–7% of agent contexts) and the net effect over all sessions in a live A/B test (about 2,500–2,900 sessions per arm at our spread).
Size	Under about $2{,}700$–$5{,}100 \times f$ tokens at $u = 0$ in contexts like ours (43 at $f = 0.8\%$); change lines only for APIs your sessions hit (7 of 25).
Guard	Fail CI on a stale block (`sync --check`; offline: `status`); review like code; `unapply` rolls back.

Table 3: Deployment checklist (release 0.4.1 commands; derivations: §5, Appendix D).

(80%; clustered CI 61–93%), 42 of them in a still-working form. None was judged to fail at runtime, but only 14 true uses were checked by running judge-written snippets (none failed), the other 34 by reading the code.

7 Lessons for Practitioners

1. Run context-file controls concurrently, and read transcripts. Our Irrelevant arm, run 3.5 hours after the others, suggested that only relevant content saves; run in the same batches, it was not detectably worse than the full block (Notes/Irrelevant tokens 0.93 [0.80, 1.10]), so we withdrew the earlier result. None runs of the same tasks differed in tokens by 18% and 15% between sittings 3.5 and 6.5 hours apart (CIs include 1 for tokens, not for None's cost over 3.5 hours), and the same Irrelevant block used 0.91 [0.82, 0.99] of its earlier tokens, so drift as large as the effect is plausible. Only transcripts and a secret scan found three silent infrastructure faults (Appendix J).

2. Size live tests for the net effect. A paired offline test (the same tasks in both arms, three runs per task and arm) detects a 15% saving with 80% power with about 5 to 50 tasks, depending on the task-level SD of the log ratio (21 tasks at our main study's SD); the exact sign-flip test needs at least 6 tasks to reach $p < 0.05$. A live A/B test over all sessions measures the net effect, u included, and is unpaired. At $u = 0$ and $f = 0.2$ the net saving is about 4–5% of a session's mean cost, and an unpaired test of mean cost on a 20/80 mix of affected and unaffected sessions needs about 2,500–2,900 sessions per arm at our spread, more in real fleets; a test on log cost would target the geometric mean, which barely moves here, and would need far more. Restricted to pre-selected affected sessions, it estimates s, not u (120–155 per arm for a 15% saving; Appendix D).

8 Conclusion: Before Rolling Out

What we can say. Two of the ten pre-committed efficiency rules held, both on one model (Opus 5.5) in one agent, on small tasks that it wrote and solved unaided, and one of them (new-task tokens) is not detected after Holm correction; the header alone gave 9–97% of the full block's cost saving, and no pre-committed test separated the two; neither detectably beat a generic warning; other models showed no detectable saving, though with the block Sonnet 5.5 passed 15.7 points more post-cutoff runs (not detected). In the author's agent contexts a real-size block costs about 0.4% of a context, a small and bounded cost, and the 0.8–7% of contexts that needed a changed API do not repay it on their own.
What we cannot say. Whether change lines matter, even on the 7 of 25 tasks they covered (post hoc; not detected without ant01); what the block does to the other contexts (u), which decides whether it pays overall (our estimate, a 6.6% saving, comes from short control sessions on self-written tasks and cannot be told from zero); whether it beats a version list or on-demand documentation (neither comparison worked); and how it behaves in large repositories. Leave it off by default, a loss-averse choice rather than a finding that it loses money, unless a live A/B test of the net effect (about 2,500–2,900 sessions per arm at our spread) shows a saving; test any block against header-only and generic arms in the same batches, and keep it under the size cap of Table 3.

Limitations

Scale and ceiling. 25 post-cutoff tasks in seven libraries for Opus 5.5, and 17 in four for Sonnet 5.5 and Haiku 5.5, limit power; most secondary effects are not significant. Opus 5.5 passed at least 92% without help, so no pass-rate claim is possible for the model where the saving appears. We did

not correct across studies or across the ten rules; Table 1 gives raw and Holm-adjusted p within each study. The exact sign-flip test conditions on our tasks, so it does not generalise to other tasks, and our per-study intervals treat tasks as the only source of variation, although None's token use drifted between sittings and the main study and replication saved on different tasks (Lesson 1; Appendix K). The pre-committed efficiency metric changed from a four-metric disjunction (main study) to total tokens (later studies).

Task provenance. All 35 tasks, reference solutions and hidden tests were written and reviewed by Claude Code sessions running Opus 5.5, the model under test, and no human has checked any of them (§4, Appendix A). The original post-cutoff tasks all come from LLM and agent SDKs, five of 17 on the evaluated vendor's SDK; without those five the replication's point saving fell from 15% to 6% (cost; Appendix D).

One agent shows the effect. Ten of the eleven completed studies use Claude Code at effort "medium"; the one completed open-weight study differs in model, scaffold and cutoff, and a quarter of its sessions ended on a fault of our adapter; a second open-model study with a fixed adapter was started and stopped before completion, and its partial runs are not analysed (Appendix F). Other commercial agents that read AGENTS.md were not evaluated. The mechanism is not established: on Opus the full block coincided with fewer reads of installed sources and the header alone did not (Appendix D), and agents with other exploration policies, models or effort levels can differ (Khatri, 2026). Which cutoff to use for an AGENTS.md read by several agents is untested. The problem is not specific to Python (Liang et al., 2025), but the tool is.

Evaluated setting and task ecology. All blocks used the imported scope with five lines per package, not the tool's default. The tasks are small repositories built so that every post-cutoff session needs a changed API; whether the relative saving holds on long sessions in large repositories, with real blocks about twice the benchmark's size, is untested. So are the break-even's assumptions (one cache write per context; s and u independent of block size and context length; u, measured in short benchmark control sessions, carried as a constant relative effect to real contexts), and u itself is bounded only loosely by our control tasks. The live A/B sizing of Lesson 2 assumes independent sessions with our benchmark's spread.

Estimate of f. It comes from the author's own sessions, in work heavy on AI libraries that includes building this tool, so exposure is likely above typical; other developers, agents and languages may differ. Installed versions are approximated by the latest release at the session date, imports could not be recovered for 23% of Python edits (imputing them raises f_{upper} to 11% of all contexts and 29% of Python-editing ones, with intervals that are too narrow), and the matcher-based f_{strict} is a lower bound. Whether contexts that import a changed package without using a changed API save was not tested (Appendix I). Its contexts, mostly subagent runs, average 45 model calls against 11 in our benchmark sessions, which raises c; K was measured in the same contexts, but whether s grows with context length is untested.

What the header tests. Each task's header names only that task's library, so no arm can test whether the diff's selection of packages matters; a benchmark with several changed dependencies per repository would. No arm named wrong packages or versions, so priming by any project-specific block (Zhang et al., 2026) is not excluded, and the concurrent study used one model, one day, eight tasks and one Generic wording.

Coverage and wrong blocks. The frozen blocks named the task's change in 7 of 25 tasks, so content was mostly untested; the covered-task saving over Header is post hoc and rests on 7 tasks, largely on ant01. The static diff does not see behaviour changes behind unchanged signatures (Chen et al., 2026), so every static tool's recall is flattered (Appendix G). Staleness hashes catch an outdated block, not an incorrect one, and the header tells the agent to follow its lines, so a wrong or incomplete note can mislead (Appendices C and E).

AI-judged evaluations. The detector ground truth, the matching of reports to it, the precision verdicts and both exposure audits are single LLM judgments (Claude Opus 5.5) without human review; matching rules were refined after seeing outputs, uniformly for all tools, and the audits measure precision only, so matcher recall is not estimated (Appendices G and H).

Cutoff anchoring. The 5.5 models share one cutoff, taken as reported by the vendor although effective cutoffs can differ (Cheng et al., 2024); we did not compare anchors on one model, so anchoring to the cutoff is a design choice, not a tested contribution.

Context7. The documentation-server arms ran without a key and hit a quota error, so we make no comparison with on-demand documentation lookup; a keyed comparison is the most useful next experiment for practitioners.

Exposure scan. The random sample covers recently active repositories with at least 200 stars and pins at the root; it excludes those without root pins and some that could not be checked out on Windows, a non-random, platform-specific exclusion. Matches cluster in few repositories, and most true uses were checked by reading code, not by running it; no project was run (Appendix H).

Author role. The author maintains the tool, and the tasks were written with models from the vendor whose agent we evaluate; pre-commitment in git, hidden tests and frozen blocks are our safeguards. Further threats (pricing, the change of efficiency metric, drift between studies, the case study, a later tool version and pilot exposure) are detailed in Appendix K.

Ethical Considerations

Data and licences. All tasks were written by AI agent sessions under the author's direction (Limitations); the libraries used are open source under permissive licences, as are the open-weight model (Apache-2.0), llama.cpp and mini-swe-agent (MIT). The case study used a subset of a public open-source repository, which we do not redistribute. The exposure scans read public repositories with shallow or sparse clones and executed no repository code (the second audit ran only snippets written by the AI judge against releases from PyPI); we report aggregates, and only the first audit file lists public repository names and paths. The tool and benchmark code are released under MIT and the benchmark data under CC BY 4.0. **Human subjects.** None. The estimate of f used the author's own agent transcripts, processed on the author's machine; only aggregate counts are reported and released. **Compute.** The 1,346 Claude sessions cost about $280 at API list prices (computed from logged token counts); the 261 sessions of the completed open-weight study ran in about 6.5 hours on one laptop GPU, with no API cost; the static scans used one laptop. **Secrets.** A maintainer's Hugging Face token leaked twice. During task construction it appeared in one agent transcript, and in 40 early main-study sessions it was readable by agent subprocesses; two of those transcripts printed it in requests to an in-memory mock. No request used it against Hugging Face, but in both cases the token entered the agent's context, which is sent to the model provider. We redacted the main-study transcripts, isolated credentials for all later sessions, and every later batch passed a secret scan with zero findings. **Risks.** Wrong notes can mislead an agent (Limitations, Appendix C); each note carries its evidence tag, and the tool never names a replacement without evidence, except a parameter it labels as a probable rename (none in our blocks). The tool never imports or executes downloaded packages. The one breaking import found by the first exposure scan is in a test server of a public repository. Context files are an injection surface (Liu et al., 2025). A note names only identifiers found in the two releases, but it can also quote up to 160 characters of a package's own deprecation message, attributed to the package, with line breaks and comment markers removed and backticks replaced; a compromised release could therefore still place instruction-like text in the always-loaded block, which should be reviewed like any other change to a context file.

AI Assistance

The experiments, analysis code and text of this report were produced with an AI coding assistant (Claude Code, Claude Opus 5.5) working under the author's direction. Claude is made by the company whose models and agent we evaluate; the assistant ran separately from the agent under study. The assistant drafted and edited the text; searched the literature; wrote much of the tool's code, the benchmark harness, drafts of the study protocols, and the analysis and plotting scripts; classified the exposure-scan matches and wrote and ran the second audit's runtime checks (§6); and built the detector-recall ground truth, matched the tools' reports to it and judged their precision (§3). Claude Code sessions running Opus 5.5 wrote all 35 benchmark tasks with their reference solutions and hidden tests, the two case-study tasks and the changed-defaults detector of the later tool version C2 (Appendix E),

and separate sessions reviewed the tasks; no human has re-checked the tasks or the audits (Limitations). AI-assisted internal reviews of drafts motivated several follow-up studies (§4). Every reported number was traced to an analysis output by script.

References

Agentic AI Foundation. 2026. AGENTS.md: A simple, open format for guiding coding agents. https://agents.md. Accessed 2026-10-22.

Anthropic. 2026. Claude Code documentation. https://code.claude.com/docs. Accessed 2026-10-22.

Ahmed Nusayer Ashik, Shaowei Wang, Tse-Hsun Chen, Muhammad Asaduzzaman, and Yuan Tian. 2026. When LLMs lag behind: Knowledge conflicts from evolving APIs in code generation. *Preprint*, arXiv:2604.09515.

Weiheng Bai, Keyang Xuan, Pengxiang Huang, Qiushi Wu, Jianing Wen, Jingjing Wu, and Kangjie Lu. 2025. APILOT: Improving the security and usability of LLM code suggestions via outdated API mitigation. In *Proceedings of the 2025 IEEE Annual Computer Security Applications Conference (ACSAC)*, pages 1193–1208. IEEE.

Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An empirical study of context files for agentic coding. *Preprint*, arXiv:2511.12884.

Jingyi Chen, Songqiang Chen, Jialun Cao, Jiasi Shen, and Shing-Chi Cheung. 2025. When LLMs meet API documentation: Can retrieval augmentation aid code generation just as it helps developers? *Preprint*, arXiv:2503.15231.

Juntao Chen, Tingting Bi, Yanlin Wang, and Patanamon Thongtanunam. 2026. Who broke me? execution-guided repair of behavioral dependency breaks. *Preprint*, arXiv:2610.09267.

Jeffrey Cheng, Marc Marone, Orion Weller, Dawn Lawrie, Daniel Khashabi, and Benjamin Van Durme. 2024. Dated data: Tracing knowledge cutoffs in large language models. *Preprint*, arXiv:2403.12958.

Keyuan Cheng, Xudong Shen, Yihao Yang, Tengyue Wang, Yang Cao, Muhammad Asif Ali, Hanbin Wang, Lijie Hu, and Di Wang. 2025. CODEMENV: Benchmarking large language models on code migration. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 2719–2744, Vienna, Austria. Association for Computational Linguistics.

Daniele Cipollone, Sergey Titov, Maliheh Izadi, Egor Bogomolov, and Arie van Deursen. 2026. LibEvoBench: Probing temporal knowledge stratification in code generation models. *Preprint*, arXiv:2606.25402. DL4Code Workshop at ICML 2026.

Xingliang Du and Jun Ma. 2022. AexPy: Detecting API breaking changes in Python packages. In *Proceedings of the 33rd IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pages 470–481.

Thibaud Gloaguen, Niels Mündler-Sasahara, Mark Niklas Müller, Veselin Raychev, and Martin Vechev. 2026. Evaluating AGENTS.md: Are repository-level context files helpful for coding agents? *Preprint*, arXiv:2602.11988.

Jian Gu, Hongyu Zhang, Chunyang Chen, and Aldeida Aleti. 2026. Mind the gaps: From failure attribution to closed-form repair of code language models. *Preprint*, arXiv:2610.05277.

Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do agent skills actually help in real-world software engineering? *Preprint*, arXiv:2603.15401.

Nizar Islah, Justine Gehring, Diganta Misra, Eilif Muller, Irina Rish, Terry Yue Zhuo, and Massimo Caccia. 2024. GitChameleon: Unmasking the version-switching capabilities of code generation models. *Preprint*, arXiv:2411.05830.

Nihal Jain, Robert Kwiatkowski, Baishakhi Ray, Murali Krishna Ramanathan, and Varun Kumar. 2024. On mitigating code LLM hallucinations with API documentation. *Preprint*, arXiv:2407.09726.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*.

Prakhar Khatri. 2026. Do context files help coding agents? a two-agent ablation study on real repositories. *Preprint*, arXiv:2607.27250.

Sachit Kuhar, Wasi Uddin Ahmad, Zijian Wang, Nihal Jain, Haifeng Qian, Baishakhi Ray, Murali Krishna Ramanathan, Xiaofei Ma, and Anoop Deoras. 2025. LibEvolutionEval: A benchmark and study for version-specific code generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6826–6840, Albuquerque, New Mexico. Association for Computational Linguistics.

Pramod Kumar and Raj Kumar Pandey. 2026. TurnCost: Turns, not observations: Cache-aware loop engineering for coding agents. *Preprint*, arXiv:2610.15093.

Man Ho Lam, Chaozheng Wang, Hange Liu, Jingyu Xiao, Haau-sing Li, Jen-tse Huang, Terry Yue Zhuo, and Michael R. Lyu. 2026. SWE-Chain: Benchmarking coding agents on chained release-level package upgrades. *Preprint*, arXiv:2605.14415.

Minh Le-Anh, Nam Le Hai, Quyen Tran, Anh Nguyen Hoang, Linh Ngo Van, Bach Le, and Nghi D. Q. Bui. 2026. Beyond repository boundaries: Cross-repository graph retrieval for code generation. *Preprint*, arXiv:2609.09987. To appear in Findings of EMNLP 2026.

Linxi Liang, Jing Gong, Mingwei Liu, Chong Wang, Guangsheng Ou, Yanlin Wang, Xin Peng, and Zibin Zheng. 2025. RustEvo^2: An evolving benchmark for API evolution in LLM-based Rust code generation. *Preprint*, arXiv:2503.16922.

Guancheng Lin, Xiao Yu, Jacky Keung, Xing Hu, Xin Xia, and Alex X. Liu. 2026. Don't use a cannon to kill a fly: Lightweight model editing for LLMs to correct deprecated API recommendations. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.

Jin Liu, Yanzhong He, Guancheng Lin, Xiao Liu, Jacky Wai Keung, Xiao Yu, and Xiaoxue Ma. 2026a. What was once learned may need to be unlearned: Machine unlearning for deprecated API knowledge in large language models. *Preprint*, arXiv:2609.25786.

Yue Liu, Yanjie Zhao, Yunbo Lyu, Ting Zhang, Haoyu Wang, and David Lo. 2025. "Your AI, My Shell": Demystifying prompt injection attacks on agentic AI coding editors. *Preprint*, arXiv:2509.22040.

Zexuan Liu, Yuning Yang, and Tiancheng Zhao. 2026b. Curating always-loaded context for LLM agents: A capacitated assortment model with censored feedback. *Preprint*, arXiv:2610.11007.

Zeyu Leo Liu, Shrey Pandit, Xi Ye, Eunsol Choi, and Greg Durrett. 2024. CodeUpdateArena: Benchmarking knowledge editing on API updates. *Preprint*, arXiv:2407.06249.

Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the impact of AGENTS.md files on the efficiency of AI coding agents. *Preprint*, arXiv:2601.20404.

Zijian Luo, Runzhi He, Pengfei Gao, Yu Kang, Zeqi Lin, Minghua Ma, Qingwei Lin, Saravan Rajmohan, and Yongqiang Tian. 2026. Update from hell: Can coding agents survive hidden breakage in dependency upgrades? *Preprint*, arXiv:2608.30300.

Yacine Majdoub, Rinad Hamid, Eya Ben Charrada, Ahmad Abdellatif, and Haifa Touati. 2026. Understanding and mitigating library-related issues in LLM-generated code. *Preprint*, arXiv:2610.00622.

Timothée Mazzucotelli. 2026. Griffe: Signatures for entire Python programs. `https://mkdocstrings.github.io/griffe/`. Accessed 2026-10-22.

Diganta Misra, Nizar Islah, Victor May, Brice Rauby, Zihan Wang, Justine Gehring, Antonio Orvieto, Muawiz Sajjad Chaudhary, Eilif B. Muller, Irina Rish, Samira Ebrahimi Kahou, and Massimo Caccia. 2026. GitChameleon 2.0: Evaluating AI code generation against Python library version incompatibilities. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 46792–46831, San Diego, California, United States. Association for Computational Linguistics.

Model Context Protocol. 2026. Model Context Protocol specification (revision 2026-07-28). `https://modelcontextprotocol.io/specification/2026-07-28`. Accessed 2026-10-22.

Matt Nigh. 2025. How to write a great agents.md: Lessons from over 2,500 repositories. GitHub Blog. Published 19 November 2025; accessed 23 October 2026.

Warren Gerald Polandra, Yoshiki Higo, and Raula Gaikovina Kula. 2026. The role of agentic instruction categories for code generation: A study of five Python libraries. *Preprint*, arXiv:2610.21421. To appear at APSEC 2026.

Vali Tawosi, Salwa Alamir, Xiaomo Liu, and Manuela Veloso. 2025. LLM agents for automated dependency upgrades. In *AISM Workshop at the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Huy Q. Tran, Dang H. Vu, Tuyen N. Dinh, Anh H. D. Nguyen, Anh N. H. Vu, Anh M. T. Bui, and Phuong T. Nguyen. 2026. Towards knowledge alignment in code LLMs: Contrastive unlearning for evolving APIs. In *Proceedings of the 42nd International Conference on Software Maintenance and Evolution (ICSME)*.

Upstash. 2026. Context7: Up-to-date docs for AI agents. `https://context7.com`. Accessed 2026-10-22.

Chong Wang, Kaifeng Huang, Jian Zhang, Yebo Feng, Lyuye Zhang, Yang Liu, and Xin Peng. 2025. LLMs meet library evolution: Evaluating deprecated API usage in LLM-based code completion. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*.

Sarel Weinberger and Amir Hozez. 2026. Prompt-induced waste in coding agents: Reasoning, effort, harness design, and end-to-end cost. *Preprint*, arXiv:2608.01347.

Haoze Wu, Yunzhi Yao, Wenhao Yu, and Ningyu Zhang. 2026. ReCode: Updating code API knowledge with reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Tongtong Wu, Weigang Wu, Xingyu Wang, Kang Xu, Suyu Ma, Bo Jiang, Ping Yang, Zhenchang Xing,

Yuan-Fang Li, and Gholamreza Haffari. 2024. VersiCode: Towards version-controllable code generation. *Preprint*, arXiv:2406.07411.

Xiaonan Xu and Wenjing Wu. 2026. Stale instruction files override the code in coding agents: A one-sentence fix. *Preprint*, arXiv:2610.30571.

Shenghao Yang, Bo Lu, Yaochen Liu, Yu Kang, Qiongfang Zhang, Chetan Bansal, Saravan Rajmohan, and Minghua Ma. 2026. DepRepair: LLM-based source-code repair for dependency breaking changes. *Preprint*, arXiv:2607.17957.

Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. 2026. Guardrails beat guidance: A large-scale study of rules, skills, and persistent configuration for coding agents. *Preprint*, arXiv:2604.11088.

A Tasks

Table 4 lists the original tasks and Table 5 the new ones.

Original task set. The author chose the libraries, the changes and the design rules (four gates, leak check, strata); Claude Code agent sessions running Opus 5.5 then wrote the tasks: one design session wrote and validated six prototypes, and five sessions, one per library group, wrote the rest, each verifying its claimed changes in the installed sources of both releases. A separate Opus 5.5 session that wrote none of the tasks reviewed every task and hidden test before the pilot, as the protocol required. It removed a module of old-API code that was broken at the pinned version from seven task repositories (hf02–hf04, pd02, pd03, lc01, lc02), because such lures show the stale answer and favour the notes condition, and it added a passing alternative solution to nine tasks; all 24 tasks passed the gates before and after. Three candidate tasks (ant06, mcp03, hf05) were dropped at a premise check during admission, before the 24 tasks were frozen and before the pilot, because the old code did not fail as assumed; the frozen protocol lists them with their probes. An example instruction (ant01): "Implement `label_ticket` in app/labeler.py. It must send exactly one Messages API request to the model in `MODEL` with max_tokens=5 [...] every request must be sent with temperature 0." Its stale pattern matches `temperature=`, `top_p=` or `top_k=` passed as a keyword.

New task set. The 11 new tasks were added to address the small number of libraries. A scan of candidate libraries with the tool's diff and runtime probes of both releases selected polars, streamlit and mistralai; fastapi and two mistralai changes were considered and dropped, because the cutoff-era code still passed, no replacement existed, or the replacement could not be verified offline. Pinned versions are the newest releases on the day the tasks were written, and the cutoff-era version is the newest release on or before the cutoff. Before any session, an addendum section fixed the study (Opus 5.5, A/C/H, three repetitions, 99 sessions, fixed seed), the sha256 of every task part and hidden test, the frozen notes and header-only blocks (released 0.4.1 with unchanged settings), and the rules: the token rule as the primary efficiency test, the pass-rate rule only below the ceiling, non-inferiority on the three controls, the stale-edit and header-only tests as before, and the pooled analysis with the 17 original tasks, with and without the pilot tasks. That the notes covered only two tasks was recorded before the runs. Tasks were written by one agent session and reviewed by another, which had no part in writing them. The reviewer checked every claimed change in the installed sources of both releases and ran 55 alternative correct and plausible wrong solutions against the hidden tests. It found two blocking problems: the pl02 tests never checked that the file was read in batches, and the pl04 tests rejected valid solutions because plain `collect()` already runs on the streaming engine in polars 2.0. Both were fixed, as were three weaker checks (st01, st90, mi02) and two over-broad stale patterns (pl01, mi02); all 11 tasks then passed the four gates and the leak check again. With the polars 2.0 default engine, pl04 in effect tests that the removed `streaming=` argument is not passed. Two runs failed: mi01 C (repetition 2) imported `httpx`; st90 H (repetition 1) bound the widget to the URL with an argument that exists in both versions but does not write the default back as the instruction asks, a borderline case for the hidden test; no C−A result changes without st90. Repositories and hidden tests of the new tasks are not published, so that they stay unseen; their definitions are in the data folder.

B Agent Configuration and Conditions

Command (prompt on standard input):

```
claude -p --model <model> --effort medium
  --output-format stream-json --verbose
  --max-turns 40 --max-budget-usd 3
  --permission-mode dontAsk
  --tools Bash,Read,Edit,Write,Glob,Grep,
```

```
        TodoWrite,Skill
  --strict-mcp-config --mcp-config <cfg>
  --no-session-persistence
```

The budget (`--max-budget-usd`) was $7.50 for Sonnet and Haiku, because the CLI priced these models at a fallback rate 2.5 times the Opus rate; this keeps the same token cap. Settings deny web fetch and search, package managers, curl, wget and `git push`; tests run with sockets limited to localhost. Base CLAUDE.md (all conditions): "Dependencies are pinned in `pyproject.toml` and already installed in `.venv` [...] Run the tests with `pytest -q`. Do not install, upgrade or remove packages, and do not change the pinned versions." Context7 rule (B, D), the server's recommended text: "Always use Context7 when I need library/API documentation, code generation, setup or configuration steps without me having to explicitly ask." Forced rule (E): look up every third-party API with `resolve-library-id` and `query-docs` before writing code that calls it, "even when you think you know the API." E was added after the pilot because B's call rate was below the pre-committed 80% threshold.

C Example Notes

Within a package, change lines are chosen in this order: APIs the code already uses, broken import paths, changes with a known replacement, other removals, parameter and member changes, and deprecations. The frozen block for ant01, a pilot-exposed task (Opus 5.5; the meta line is shortened):

```
<!-- since-cutoff:start -->
<!-- since-cutoff:meta {"v":2,"tool":"0.4.1",
  "model":"claude-opus-5-5","cutoff":"2026-06-30",
  "scope":"imported", ...} -->
## Library changes after the model's training cutoff
```

Header (kept in condition H): "Changed after the training cutoff of `claude-opus-5-5` (2026-06-30) in the packages this project imports, at the versions in `pyproject.toml` [...] Tags: [diff] a static comparison of the two releases' public APIs; [library] the replacement is named in the library's own deprecation text [...] No library code was run. Where these lines conflict with what you remember, follow these lines." **anthropic 1.8.0** (0.115.0 at the cutoff)

- `Messages.create()` no longer accepts `temperature`, `top_k` or `top_p` as keyword arguments. If the API still needs them, pass them through its `extra_body` or `extra_query` argument. [...] [diff]
- `anthropic.AI_PROMPT` was removed; do not use it. [diff]
- `anthropic.types.beta.deleted_file` moved to `anthropic.types.deleted_file`: import it with `from anthropic.types import deleted_file`. [diff + move checked]
- (two further bullets of the same kinds)

The hf02 bullet that misled Haiku 5.5, which passed hf02 in 3 of 3 runs without notes but 1 of 3 with this note (Sonnet 5.5: 3 of 3 and 2 of 3; an observation; without a confirmed replacement, pointing to the installed package may beat forbidding the old name, untested): "`huggingface_hub.upload_large_folder` was removed; do not use it. since-cutoff found no replacement in huggingface-hub's deprecation text. [diff]" Other bullets in the same block did name replacements ("`huggingface_hub.duplicate_space` was removed; do not use it. Use `HfApi.duplicate_repo` instead. [diff + library]").

D Full Statistics

The appendices use the arm letters of Table 2: A None, C Notes, H Header, R Irrelevant, G Generic; C2 is a block from a later, unreleased tool version (Appendix E); B, D and E are the Context7 arms (Appendix B). All intervals are 95% task-cluster bootstrap intervals (10,000 resamples, fixed seed); ratios are post-cutoff unless stated and, as in Table 1, are geometric means over tasks of per-task ratios of median run values, except Figure 3; "w/o pilot" excludes ant01 and mcp01; p-values are rounded to two significant figures, and an interval bound that would print as 1.00 is given with more digits where it matters. A *stale first edit* is a session whose first edit of a target file matches a task-specific pattern for the cutoff-era API (pre-committed secondary outcome).

Table 12 lists every pre-committed efficiency rule, Tables 7 and 8 the statistics behind §5, Table 9 the exploration counts and Figure 2 the per-study ratios.

Token types, turns and cost per solved task (exploratory). Token types come from the CLI's cumulative usage record; list prices for Opus 5.5 are $4 (input), $20 (output), $0.20 (cache read) and $8 (1-hour cache write) per million tokens. In

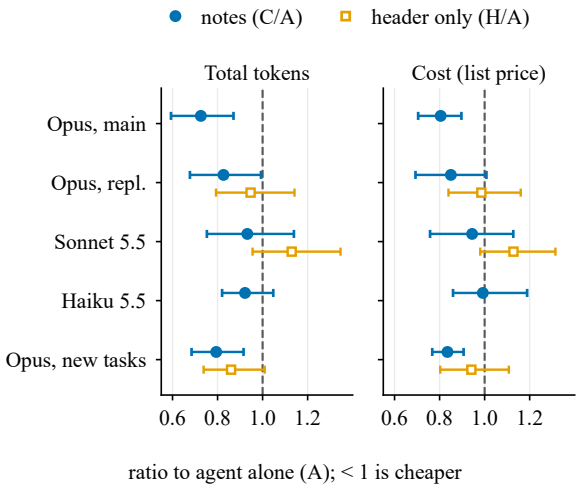


Figure 2: Ratios to the agent alone on the post-cutoff tasks (17 per original study, 8 new; 95% task-cluster bootstrap CIs). Filled circles: notes (C/A). Open squares: header-only control (H/A), shown for three of the five studies with H (the other two: Appendices E and F).

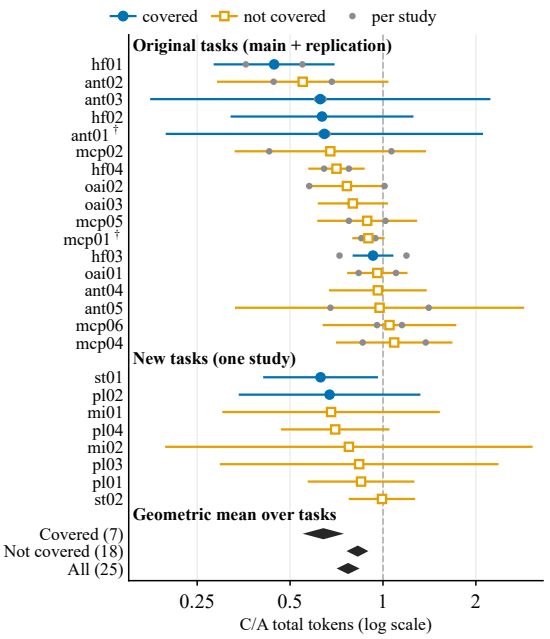


Figure 3: C/A total tokens per post-cutoff task, Opus 5.5 (log scale; per-task ratios of geometric means over repetitions, unlike the median-based ratios elsewhere, with 95% intervals, wide by design). Original tasks combine the main study and the replication (grey: each study; some grey dots are hidden behind markers). Blue: the notes named the task's change (a post hoc split for the original tasks); orange: they did not; †: seen in the pilot. Diamonds: geometric means over tasks with task-bootstrap CIs.

every Opus study cache reads are about 90% of A's tokens (main 90.2%, replication 89.7%, new tasks 90.4%) but 15–16% of its cost; cache writes are 52–61% of cost. Of the C/A token saving, cache reads are 93% in the main study, 86% in the replication and 93% on the new tasks. With total tokens decomposed per task into turns times tokens per turn, turns explain 62% [49, 81] of the main study's token saving, 97% on the new tasks and an undetermined share in the replication (50% [−9, 118]); for cost they explain 80% [64, 96] in the main study and an undetermined share in the replication (44% [−3, 103]), consistent with turn count driving agent cost (Kumar and Pandey, 2026). Per-task token and cost ratios correlate at r = 0.84 (main), 0.95 (replication) and 0.96 (new tasks); they disagree in sign on 2 of 17 main-study tasks. Cost per solved task, C/A: main 0.76 [0.63, 0.86] (w/o pilot 0.80 [0.73, 0.88]); replication 0.76 [0.60, 0.89] (w/o pilot 0.82 [0.73, 0.91]; H/A 0.88 [0.76, 1.01]); new tasks 0.82 [0.75, 0.91]; pooled 0.78 [0.68, 0.86] and 0.78 [0.66, 0.89]; Sonnet 5.5 0.67 [0.47, 0.89]; Haiku 5.5 0.93 [0.75, 1.09]; Sonnet 4.5 1.05 [0.83, 1.45]; concurrent study 0.77 [0.70, 0.85]. Because cost per solved task equals the summed cost ratio times A's passes over C's passes, the main study's 0.76 is its summed cost ratio, 0.80, times 48/51 (A's three ant01 failures); the replication's summed ratio is 0.82. The notes block adds about 7.3k cached tokens per C session in the main study (1.9% of A's tokens; replication 2.5%, new tasks 3.1%, case study 6.5%): \$0.0015 of cache reads plus at least one cache write of the block (about 440 tokens on average over the 17 original post-cutoff tasks; 423 over all 20 non-empty original blocks), \$0.0035, about \$0.005 in all. Exact sign-flip tests on per-task log ratios (cost; tokens; raw p): main p = 0.00027; 0.0033; replication 0.13; 0.074; new tasks 0.0078; 0.031; concurrent C/A 0.016; 0.031, H/A 0.031; 0.016, R/A 0.13; 0.078; pools main + new $p < 0.001$; <0.001, replication + new 0.0087; 0.0072. With Holm correction over C/A, H/A and C/H, the new tasks' token p becomes 0.094 and their cost p 0.023; Table 1 gives the raw and Holm-adjusted p of every study's pre-committed metric. In log cost, the header alone (H/A) recovered 9% of C's saving in the replication, 34% on the new tasks, 97% in the concurrent study, 17% in the pre-committed replication + new pool and 32% in the post hoc 25-task pool below (tokens: 29%, 65%, 114%, 42%, 53%). Above 100%, the header saved more than the full block. Coverage

split (post hoc and exploratory; median-based, as Table 1). C/H tokens on the tasks whose block named the change against the others were 0.59 (5 tasks) and 1.03 (12) in the replication (exact permutation p = 0.022; 0.84 without ant01, p = 0.17), 0.82 and 0.96 on the new tasks and 0.89 and 1.09 in the concurrent study (2 and 6 tasks; with 28 label assignments no p below 0.036 is possible). In the post hoc 25-task pool below, C/H tokens were 0.66 [0.39, 0.91] on the 7 covered (sign-flip p = 0.016) and 1.03 [0.94, 1.13] on the 18 other tasks, a ratio of 0.64 [0.38, 0.91] (stratified task bootstrap; exact two-sided permutation of the coverage labels over all 480,700 assignments, p = 0.0058); in cost 0.69 and 0.98, ratio 0.71 [0.46, 0.96], p = 0.021. Without ant01, on which the tool was changed after the pilot (C/H tokens 0.15 in the replication), the token ratio is 0.82 [0.71, 0.95], p = 0.051, and the cost ratio 0.87 [0.75, 0.996], p = 0.15; without both pilot tasks 0.83, p = 0.059 (tokens). So under our rule (§4) the split is not detected without ant01; dropping any other covered task instead leaves p between 0.0034 and 0.018 (tokens; cost 0.014–0.045). Counting the partly covered mcp01 (Table 4) as covered gives 0.68 [0.41, 0.94], p = 0.012 (cost 0.74, p = 0.029). Every covered task but ant01 had pooled C/H tokens of 0.66–1.00, but per study the direction was not stable: in the new-task study pl02 went the other way (C/H tokens 1.07) and in the concurrent study st01 did (1.17), so each of these studies' covered-task saving rests on one of its two covered tasks. H/A showed no such gap (tokens 0.95 and 0.87, ratio 1.09 [0.83, 1.55], p = 0.57; cost p = 0.78), so the gap in C/A (tokens 0.63 and 0.89, ratio 0.70, p = 0.020; 0.78, p = 0.085 without ant01) points to the change lines rather than the header. In the main study alone, which had no H arm, covered and other tasks saved alike (C/A tokens 0.74 and 0.72, p = 0.87). The permutation test treats tasks as exchangeable, but covered tasks also differ in library and difficulty, and per-task effects did not replicate between main study and replication (below). Equivalence (post hoc; margin ±10%, C/H within [0.90, 1.10]; equivalent at 5% if the 90% task-bootstrap interval lies inside, checked with exact one-sided sign-flip tests of the log ratios shifted to each bound): in the 25-task pool C/H cost was 0.89 (90% CI 0.78–0.98, TOST p = 0.56; sign-flip TOST 0.56) and tokens 0.91 (0.77–1.02, p = 0.44), so only differences above about 22% are excluded; in the pre-committed replication + new pool cost was 0.87 (95% CI 0.74–0.99, two-sided sign-flip p = 0.051, so not detected by our rule; TOST p = 0.66). Per study, only the concurrent study's cost lies within the margin by the bootstrap (0.99, 90% CI 0.91–1.09, TOST p = 0.036), not by the sign-flip test (p = 0.074); its tokens do not (1.04, 0.93–1.16), nor do replication and new tasks (cost 0.86 and 0.89). Without the pilot tasks the pooled tokens are equivalent by both tests (0.97, 90% CI 0.91–1.05; p = 0.031 and 0.044), cost is not (0.94, 0.88–1.01, p = 0.14). With the log-symmetric margin [0.91, 1.10] no comparison is shown equivalent. At ±15% ([0.85, 1.176], the margin that our protocol pre-committed for the related comparison of Header with a lockfile-only header), both pools still fail in cost and tokens (TOST p = 0.27 and 0.22 in the 25-task pool, 0.35 and 0.29 in the replication + new pool), while the concurrent study (cost and tokens) and both pools without the pilot tasks lie within it, by both tests except for the replication + new pool's cost without the pilot tasks (bootstrap only; sign-flip p = 0.051). H/A cost was 0.99 [0.84, 1.16] in the replication, 0.94 [0.80, 1.11] on the new tasks and 0.80 [0.69, 0.92] in the concurrent study (sign-flip p = 0.86, 0.48, 0.031). Post hoc pool of all randomised H evidence (25 tasks: the 17 original tasks from the replication, each new task's log ratio averaged over the new-task and concurrent studies): H/A cost 0.95 [0.85, 1.06], p = 0.38, tokens 0.89 [0.79, 1.02], p = 0.11; C/A cost 0.84 [0.72, 0.95], p = 0.0058; C/H cost 0.89 [0.76, 1.0004], p = 0.083, tokens 0.91 [0.75, 1.04], p = 0.30. On the 8 new tasks alone (two studies averaged) C/H cost was 0.94 [0.90, 0.99], p = 0.039 (one of two C/H contrasts that meet our rule, with the covered tasks above; both post hoc, outside any family), and tokens 0.98, p = 0.70; without the pilot tasks (23) H/A tokens were 0.86 [0.76, 0.96], p = 0.018, and cost 0.93, p = 0.30. Cache writes per post-cutoff session, A / C / H: replication 26.6k / 20.4k / 25.0k, new tasks 12.5k / 10.0k / 11.7k, concurrent study 14.1k / 11.2k / 11.3k. C cut cache writes in every study and H clearly only in the concurrent study, which is why H's cost share is smaller than its token share. Header break-even (cost model of §5 at $u = 0$, s and c of each arm on the same tasks and sessions): the header alone, at its benchmark size (244 tokens on average; it names one package, so a real header is no smaller), saved $0.021 gross per affected session in the pre-committed replication +

new pool and \$0.025 in the post hoc pool, 33% and 39% of the full block's \$0.062 and \$0.063, and, with every length in model calls, breaks even at f^* = 12.2% and 10.2% (bootstrap upper bound: never; in 17.3% and 11.8% of draws its saving does not cover its cost), against 11.7% and 11.4% for the full block at 704 tokens (never-paying draws: 0) and 7.1% and 7.0% at its benchmark size; the header's f^* exceeded the full block's at 704 tokens in 52% and 43% of draws. By study: replication 12.4% against 10.6%, new tasks 11.8% against 15.1%, concurrent study 5.3% against 13.0%. These are benchmark-length values; at the length of real agent contexts every break-even is higher (Table 20). The cost evidence therefore favours neither configuration at real size, and the header's saving is the less certain. On the 17 original tasks, per-task C/A token ratios combined over the main study and replication give 0.78 [0.69, 0.87]; over all 25 tasks 0.77 [0.71, 0.84], 0.77 [0.70, 0.84] without the pilot tasks (geometric means over repetitions, as in Figure 3; the median-based estimate of Table 8 is 0.78 [0.71, 0.86]). On the 7 tasks whose block named the change the combined ratio is 0.64 [0.55, 0.75] and on the other 18 it is 0.83 [0.76, 0.90] (median-based 0.65 and 0.84; post hoc for the original tasks). The two runs of the 17 original tasks did not agree task by task: per-task log C/A ratios of the main study and the replication correlate at $r = -0.19$ for cost (Spearman -0.22) and -0.24 for tokens, and the cost ratio is below 1 in both runs for 9 of the 17 tasks.

Block size and break-even (exploratory). In the random sample (Appendix H), the median non-empty imported-scope block has 704 tokens in G and 1,025 in AI-stack repositories (means 1,014 and 1,320); at benchmark length, with the main study's saving and model calls, the break-even share is 8.3% and 12.1% at the medians and 11.9% and 15.5% at the means, and with the new tasks' saving and model calls 15.1% and 21.9% at the medians and up to 28.2% at the AI-stack mean (point estimates by the formula below; at the contexts' length the range is 13.8–49.4%, Table 20). The rest of this paragraph uses the earlier convenience sample. We ran the tool's dry run (version 0.5.0, the benchmark's settings: imported scope, five lines per package; static only, nothing written) on the 47 scanned repositories. It would write no block in 16 and a median of 764 tokens in the other 31 (IQR 574–1,329; 4 packages, 14 change lines; largest 11,100 tokens with 69 packages); in the tool's default scope, which keeps only changed APIs the code is matched to use, 10 repositories get a block (median 303 tokens). Over the 38 repositories with a post-cutoff pin, the imported-scope header names a median of 3 packages [IQR 1–6] against 9.5 post-cutoff direct pins [5–22.5] (222 of 847 in total; 7 of the 38 get no block); the named packages can include imported transitive ones, so the difference is an approximate count of what the diff filters out. For A/B test sizing, the task-level standard deviation of the log C/A cost ratio was 0.26 in the main study, 0.41 in the replication and 0.13 on the new tasks; with $n = ((1.96 + 0.84)\,\mathrm{SD}/\ln(1/0.85))^2$, detecting a 15% saving with 80% power needs about 21, 50 and 5 tasks with three runs per arm. This normal approximation assumes a paired design, the same tasks run in both arms, three runs per arm and our agent, model and effort; the exact two-sided sign-flip test needs at least 6 tasks to reach $p < 0.05$, and an unpaired A/B test on live sessions needs far more (below). At 764 tokens and the main study's saving and model calls, 1,000 benchmark-length sessions a day of which a fifth need a changed API pay \$7.84 a day for the block (\$0.0078 per session) and save \$17.45 gross (\$0.087 per affected session), a net \$9.61 a day; at 704 tokens (the random sample's median non-empty block) the figures are \$7.22, \$17.45 and \$10.22. On a matched base, 1,000 agent contexts a day like those behind f pay \$12.00 a day for a 704-token block and, at $u = 0$, save \$0.73 gross at f_{strict} (0.84%) and \$6.07 at f_{upper} (7.0%), a net loss of \$11.27 or \$5.93 a day. Per session the block costs its size times (model calls $\times$ \$0.20 + \$8) per million tokens at Opus 5.5 list prices (a cache read per model call, one one-hour cache write). With the main study's model calls this is \$0.0043 at 423 tokens, \$0.0078 at 764 and \$0.0136 at 1,329. Assuming the saving per affected session does not change with block size, the break-even share of sessions that need a changed API is the overhead over the gross saving: at 764 tokens 9.0% [6.5, 14.4] with the main study's saving, 10.4% [7.9, 15.2] with the main + new pool and 16.4% [12.2, 25.0] with the new tasks' (each with its own saving and model calls; joint task bootstrap of saving and benchmark overhead); at 423 tokens 5.0%, 5.8% and 9.1%; at 1,329 tokens 16–28%. The replication's saving (\$0.069) and model calls give 11.5% [7.0, 26.4] at 764 tokens. Fleet cost follows the mean block, not

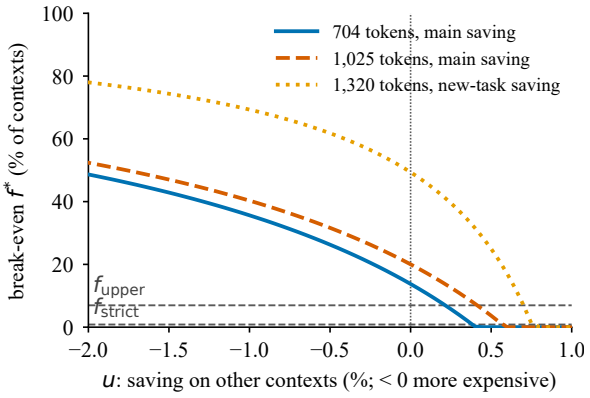


Figure 4: Break-even share f^* of agent contexts that need a changed API against the block's relative effect u on the cost of the other contexts ($u > 0$: cheaper), in one unit (model calls): at the mean length of the author's agent contexts (45.3 calls), with K = \$2.96 measured from their logged usage, for three real block sizes and savings. Dotted: $u = 0$. Dashed: the author's f_{strict} (0.8%) and f_{upper} (7.0%) over all agent contexts, the same base. A curve reaches 0 at $u = c/K$ (0.40%, 0.59% and 0.76%), from where the block pays at every f. The control-task estimate of u, 6.6% (95% CI −3.6% to +15.6%), spans the whole horizontal axis.

the median: at the mean non-empty block (1,306 tokens; \$0.0134 per session with the main study's model calls) the break-even share is 15.4% (main), 19.6% (replication) and 27.9% (new tasks), each with its own model calls. The 11,100-token block never breaks even at these savings.

Two-parameter break-even: details. The model of §5 adds the block's effect u on sessions that need no changed API: net saving per session $fs+(1-f)\,uK-c$, so $f^* = (c-uK)/(s-uK)$, and as the net is linear in f the block pays at every f once $u \geq c/K$, given $s > c$ (true for every block size here but the 11,100-token one, whose benchmark-length c = \$0.114 exceeds s = \$0.087; Figure 4). Per point, u is worth $(1-f)K$ and f is worth $s-uK$: a ratio of 2.0 at $f = u = 0$ with a benchmark control session's K and the main study's saving, and a far larger one with the contexts' measured K (\$2.96), which is why u dominates the decision in real contexts. We take t as the mean number of model calls (distinct assistant messages, the count used for the contexts) of a treated post-cutoff session: 11.3 in the main study, 12.0 in the replication and 8.8 on the new tasks (the CLI reports 16.6, 17.6 and 12.2 turns, which count tool round trips); at the 8.4 calls of control sessions the 704-token block costs \$0.0068 rather than \$0.0072. The benchmark block's own cost inside s stays priced over CLI turns (over model calls s would be 0.5% lower). The contexts in which f was measured average 45.3 model calls, so there the block costs \$0.0120 and every f^* is higher (Table 20); only these per-context values should be compared with the measured f. We take $u = 1-r$ for the C/A ratio r on the ten pooled main + new-task control tasks, 0.92 [0.79, 1.05] in tokens and 0.93 [0.84, 1.04] in cost (point estimates u = 8.3% and 6.6%), and, at benchmark length, K = \$0.17, the mean list cost of their 30 A sessions (8.9 model calls each); in the contexts behind f, K is measured directly (below). As r includes the benchmark block's own cost (1.2% of K), $1-r$ understates u slightly. At benchmark length, with the main study's saving and the 704-token block, f^* is 8.3% at $u = 0$ (the reference), 14.5% at the cost bound $u = -3.7\%$ and 17.0% at the token bound -5.3%; the block pays at every f from u = 4.2%, a value inside both intervals but, as the identical-context controls below show, within their noise. Over the four savings (main, replication, new tasks, main + new) and four block sizes (medians and means of both strata), f^* at the cost bound is 14.5–37.0% (token bound: 17.0–40.2%), and the block pays at every f from u = 4.0–8.0%. Real agent contexts are longer and dearer: with K measured in the contexts behind f (\$2.96 over all of them, \$5.97 over the Python-editing ones), c/K is only 0.40% and 0.29%, and the u the 704-token block needs at the measured f is 0.38% at f_{strict} and 0.22% at f_{upper} over all contexts, and 0.26% and 0.04% among Python-editing ones (unified analysis below). Only this matched base, where f, c and K come from the same contexts, should be used for the decision. At benchmark length with K = \$0.42, the mean None cost of a post-cutoff main-study session, every effect of u grows: the block pays at every f from u = 1.7%, and f^* is 22% at $u = -3.7\%$. The interval for r is mostly noise: on the four controls whose block was empty (oai05, pd01–pd03), where C and A get the same CLAUDE.md, r was 0.94 in tokens and 0.95 in cost, with per-task token ratios from 0.56 to 1.33. On the six controls whose block was non-empty, r was 0.91 [0.81, 0.997] in tokens (bootstrap only) and 0.92 [0.82, 1.03] in cost (K = \$0.18); at the cost bound ($u = -3.4\%$) f^* is 14% for the 704-token block (benchmark length) (six tasks; sign-flip p = 0.16 and 0.31); the ten-task pool stays the main range. The downside is the plausible direction: context files can raise cost by

more than 20% (Gloaguen et al., 2026), and in the concurrent study Generic used 1.55 times None's tokens on its three controls (Table 14). Measuring u directly is impractical: in real contexts c/K is 0.4%, and even at benchmark length telling $u = 0$ from $u = c/K = 4.2\%$ ($\delta = -\ln(1 - 0.042)$, unpaired test of log cost) needs about 500 sessions per arm at the SD of log cost of None control sessions (0.24, 30 sessions) and 1,700–2,220 at the post-cutoff SDs (0.45, 0.51), more in real fleets. A live A/B test of the net effect (below) is the practical measurement.

Unified-unit break-even with uncertainty (post hoc). Here every length is in model calls, and K is measured in the contexts behind f: their logged usage (input, output, cache reads and both cache-write durations per message), priced at Opus 5.5 list prices whatever model ran the context, gives \$2.96 [1.49, 5.90] per context over all 1,308 contexts and \$5.97 [2.79, 9.14] over the 513 Python-editing ones, so the 704-token block's c (\$0.0120 and \$0.0171) is 0.40% and 0.29% of K. Affected contexts cannot be removed from K, because the per-context flags behind f were not kept. A joint bootstrap (10,000 draws) resamples the 170 project clusters behind f (so f, the contexts' calls and K move together), the main study's post-cutoff tasks (for s and the benchmark's calls) and the ten control tasks (for u), independently. Its s prices the benchmark block's own cost over model calls (\$0.0868 rather than \$0.0872), which is why its Python-editing break-even is 19.7% rather than the 19.6% of Table 20. Table 13 gives the net saving per 1,000 contexts. At $u = 0$ the block loses money at both measured f over all contexts; at the control-task u it gains in 89–91% of draws in every cell, with intervals that span zero. The break-even share at $u = 0$ is 13.8% [9.4, 23.8] over all contexts and 19.7% [13.5, 31.6] among Python-editing ones, and the u needed at the measured f is 0.38% [0.25, 0.66] at f_{strict} and 0.22% [0.02, 0.51] at f_{upper} over all contexts. If the saving instead grew in proportion to context length, the block would pay at f_{upper} even at $u = 0$, over all contexts in 85% of draws and among Python-editing ones in all of them. With K taken as the benchmark's control cost scaled by calls (\$0.88) instead of measured, $P(N > 0)$ at the control-task u is 0.85–0.90; no K can raise it above $P(u > 0) = 0.90$. No configuration with a fixed saving per affected use reaches $P(N > 0) \geq 0.95$, so a rule that enables the block only when the one-sided 95% lower bound of N is positive leaves it off; that is a loss-averse rule, not evidence of a loss.

Random effects, block-size cap and live A/B sizing (exploratory). A DerSimonian-Laird random-effects model of C/A cost over the four Opus studies (per-study log ratio, standard error $\text{SD}/\sqrt{\text{tasks}}$; main 0.80, replication 0.85, new tasks 0.83, concurrent 0.79) gives 0.82 [0.77, 0.87], with Q = 0.65 on 3 degrees of freedom and an estimated between-study variance of 0; the studies share tasks (main and replication; new tasks and concurrent), so they are not independent and the interval is approximate. Solving the break-even for b gives the largest block that pays for itself, $b_{\max} = f\,s/(t\,p_r + p_w)$: about $8{,}500 \times f$ tokens at benchmark length with the main study's gross saving and model calls (850 at f = 0.1, 1,700 at 0.2), $6{,}600 \times f$ with the replication's and $4{,}700 \times f$ with the new tasks'. At the contexts' length (45.3 calls) the coefficients are about 5,100, 4,100 and 2,700: the cap is about 43 tokens at f_{strict} (0.84%) and 360 at f_{upper} (7.0%); among Python-editing contexts (81.6 calls), 77 at 2.1% and 640 at 17.7%. A live A/B test over all sessions estimates the mean cost per session, so we size an unpaired two-sided 5% test of mean cost with 80% power, $n = 2\,(1.96 + 0.84)^2\,V/\delta^2$ sessions per arm, on a 20/80 mix of affected sessions (the 51 main-study post-cutoff None sessions: mean \$0.4193, SD \$0.1654) and unaffected ones (the 30 control None sessions: mean \$0.1717, SD \$0.0413). The mix has mean \$0.2212 and SD \$0.1290, larger than either group's because their means differ. At $u = 0$ and f = 0.2 the expected net saving is $\delta = 0.2\,s - c$ = \$0.01013 per session, 4.58% of the mean, with s and c in model calls (704-token block, main-study saving); this needs about 2,546 sessions per arm, and 2,912 if the block is priced over CLI turns (δ = \$0.00947). A test on log cost would target the geometric mean, whose change here is only 0.56% (unaffected sessions 4.6% dearer, affected 18.9% cheaper), and would need about 101,000 sessions per arm. Restricted to affected sessions (session-level SD of log list cost of None sessions on post-cutoff tasks: 0.45 in the main study, 51 sessions, and 0.51 over the four Opus studies, 150), a 15% saving needs about 120–155 sessions per arm and a 20% saving 63–82. In practice the restriction must be made before a session starts (for example by the files its task touches); selecting on

the agent's own edits would condition on a variable the block can change and bias the estimate, and whether a pre-session selection keeps most of the effect is untested. These figures are illustrative: they assume independent sessions with our benchmark's spread, and real sessions vary more, which raises every number in proportion to the variance.

Without the vendor's SDK (exploratory). Excluding ant01–ant05 (the only tasks that use the anthropic SDK; none is a new task or a control), C/A on post-cutoff tasks was: main study (12 tasks) tokens 0.70 [0.54, 0.87], p = 0.0078, cost 0.75 [0.64, 0.85], p = 0.00049; replication tokens 0.89 [0.77, 1.05], p = 0.19, cost 0.94 [0.85, 1.04], p = 0.26; pool main + new (20 tasks) cost 0.78 [0.70, 0.85], $p < 0.001$; pool replication + new cost 0.90 [0.83, 0.96], p = 0.011; the same holds without the pilot task mcp01. On the five anthropic tasks alone the main-study cost ratio was 0.96 and the replication's 0.67, where A failed 4 of 15 runs. In the replication + new pool without them, H/A tokens were 0.87 [0.80, 0.96] (p = 0.012) and C/H 0.98 [0.88, 1.08], against C/H 0.61 [0.29, 0.97] on the five anthropic tasks. All p are exact sign-flip, unadjusted.

Exploration (exploratory). On Opus, notes cut *package-source reads* (Read, Grep or Glob calls on installed packages) from 5.9 to 4.2 per session (main study, unadjusted p = 0.018); on Sonnet the header alone raised them (2.7 to 3.6) and notes lowered them (2.2). No contrast survives Holm correction (Table 9). In the concurrent study, reads per post-cutoff session were A 2.96, C 2.00, H 2.88, R 2.21 and G 3.71. List-price bills, A to C, over all sessions: replication $18.26 to $14.99 (51 each), new tasks $6.27 to $4.97 (33 each); C/A wall time: new tasks 0.91 [0.85, 0.99], p = 0.078, replication 0.94 [0.82, 1.06]; C/A turns, sign-flip p: main 0.00089, new tasks 0.016, replication 0.94 [0.88, 1.01], p = 0.15; main wall time p = 0.0070. Pre-committed decision outcomes: main study, met for "notes don't hurt" and "notes reduce correction attempts, time or cost" (cost, turns and wall-time CIs below 1, in both analyses; failing test runs 0.14 per session in both arms, no difference; the rule is a disjunction over four metrics, and the cost CI stays below 1 at the Bonferroni level of 98.75%, [0.68, 0.92]); not met for any pass-rate or Context7 claim (ceiling, quota). Every non-inferiority rule reported as met here rests on a bootstrap bound that is degenerate when no control run failed; exact one-sided Clopper-Pearson bounds on C's control pass rate (runs treated as independent, which flatters clustered runs; A passed every control run, so the bound minus 100% bounds the difference) are 86.7% for 21 of 21 runs and 71.7% for 9 of 9, so no single study excludes a 10-point drop, while main + new tasks give 90.5% (30 of 30; 84.7% over the 18 runs with a non-empty block) and the five 5.5-model studies 93.6% (45 of 45 non-empty-block runs). Sonnet and Haiku: token rule not met; pass-rate rule (bootstrap CI above 0 and Holm $p < 0.05$) not met (on Sonnet the CI excluded 0 but Holm p = 0.19); non-inferiority met. Replication: the rule needed both the token and the cost CI below 1; tokens below 1, cost not, so not met; neither holds without the pilot tasks. Header-only rules (change lines add to the header; the header alone gives the gain): neither met in any of the three Claude studies that pre-committed them (Sonnet 5.5, replication, new tasks; the concurrent study pre-committed other contrasts) (on Sonnet 5.5 the first held for turns over all tasks but not without the pilot tasks). New tasks: token rule met (cost, turns and wall-time intervals also below 1); pass-rate rule not applicable (ceiling); non-inferiority met; stale-edit rule not met (no stale first edits without notes). Pooled analyses (secondary): token and cost intervals below 1 in both pooled sets, with and without the pilot tasks; no pooled pass-rate or stale-edit test is significant.

Context7 (exploratory). The agent called Context7 in only 10 of 51 post-cutoff B sessions (19.6%; 15.6% without the pilot tasks; 23 of 72 over all tasks). All B, D and E sessions ran Context7 without a key, and from the 215th of 360 sessions every call returned a quota error that came back as a normal tool result. Some documentation was received (transcript count: a Context7 result other than an error or the quota message) in B in 14 of 23 calling sessions, in D in 8 of 13 and in E in 42 of 72; documentation matching every answer key of the task (the per-run flag `doc_hit`) in 7, 3 and 32. With forced lookups (E), E/A cost was 0.85 [0.71, 0.98] (bootstrap only), stale first edits occurred in 1 of 51 runs, and C was not significantly cheaper than E (0.95 [0.78, 1.19]). On the 34 post-cutoff E sessions that received any documentation (27 by `doc_hit`), C and E both passed 34 of 34 (session-level C/E cost 0.86 [0.69, 1.04]).

E Follow-up Studies

The four follow-up studies were pre-committed in two addendum sections and the concurrent study in a third, each committed and tagged before its first session, with the same agent settings, grading and validity rules as before (Table 14). A fourth section (14) pre-committed a lockfile-only header L (the project's pinned direct dependencies with their versions and H's "follow these lines" sentence, without cutoff versions, diff or change lines; about 56 tokens), to run concurrently with A, C and H on the new tasks and with A and H on the 17 original tasks, and an optional wrong-block arm W (H plus one false change line). It was registered but not run: its first launch failed before any session reached the model because the Claude CLI login had expired, no L or W session ran on Claude, and we report no result. The protocol, frozen blocks and analysis script are in the data folder. Pre-committed outcomes: R, "content matters" (C/R tokens CI below 1) met and "any change list gives the gain" (R/A CI below 1) not met, both superseded by the concurrent study; older cutoff, the pass-rate, token and stale-edit rules not met; C2, pass-rate and stale-edit non-inferiority met and the efficiency rule (C2/C upper bound at most 1.10) not met; case study, descriptive only; concurrent study, C/G not significant (CI below 1 but Holm $p = 0.25$), C/R not supported (earlier R result not replicated), G/H no difference shown by the rule.

Concurrent five-arm study. All five arms ran on the 11 new tasks in the same randomised batches (task and repetition, arm order shuffled within each batch), three repetitions, one harness version and one sitting of about 95 minutes on 23 October 2026. A, C, H and R used the same frozen CLAUDE.md texts as in the new-task and R studies; only the run order was new. G is one paragraph, the same for every task, of about 218 estimated tokens: "Your knowledge of library APIs may be out of date. [...] Before using an API from an installed package, check its signature in the installed source [...] or with `help()` [...] Where the installed code conflicts with what you remember, follow the installed code." It names no package, version, model, cutoff or tool. The family was C/G, C/R and G/H on total tokens over the eight post-cutoff tasks, Holm-adjusted. Non-inferiority by the strict 10-point rule failed for R (one mi01 run imported `httpx`, the failure seen before in C; one st90 control run hit the borderline hidden test seen before in H) and, for stale first edits, for H and R (one run each). Another workload shared the machine during this study, which may affect wall time but not tokens.

Irrelevant-content control R (superseded). This study's C/R result did not replicate when R ran concurrently with A, C and H (above); we keep its description for completeness. Each R block is the task's H block plus one package section, copied verbatim from the frozen main-study notes (same model and cutoff), for a package the task does not use and whose text contains none of the task's note keys or answer tokens; the donor (mcp, openai or huggingface-hub) was chosen to make R's length closest to C's. The header still says the lines concern the project's own packages, and the donor packages are not installed. R ran about 3.5 hours after the A, C and H runs of the new-task study (started 21:08 against 17:32 UTC on 22 October); one concurrent A run per task estimates drift, and the pre-committed rule reports a confound only if its CI excludes 1, which it did not. Only C reduced package-source reads; one of 33 R transcripts mentions a donor package.

Older-cutoff model. Notes were regenerated with 0.4.1 for Sonnet 4.5's documented cutoff (2025-07-31), comparing, for example, openai 1.98.0 and polars 1.31.0 with the pinned releases. Fifteen of the 23 post-cutoff tasks without pilot exposure were drawn by a fixed seed (no controls). Six tasks differed in passes (four lower with notes, two higher). In oai01 two of three C runs imported `httpx` and failed, while the A runs found `httpx2`; its block listed five other removed openai names and not the switch. The study tests whether notes anchored to this model's cutoff help this model; it does not compare anchors on one model.

Case study. The repository is LiteLLM at a commit of 21 October 2026, the only audited repository with old-form uses; the agent worked in a 193-file subset with the real `pyproject.toml` and `uv.lock`, and only mcp 2.3.0 and its dependencies were installed. Of its five old-form uses, the two in the MCP test server became tasks: cs01 (the stdio server crashes on start) and cs02 (HTTP mode must serve at `/mcp`, which also needs a host/port change the audit and the notes missed). The other three were dropped before any session: an `httpx` client still works at runtime with mcp 2.3.0, and two subscription calls only warn and need the LiteLLM proxy. After the study we found that the cs01 stdio

smoke test closes standard input at once, so the server can exit before it answers; even the reference solution failed 2 of 8 repeated runs. All six failing cs01 runs (four A, two C) failed only this check. A corrected test that keeps standard input open (gates re-checked) passes all ten cs01 runs, three of three repeats each; we report these grades as exploratory and the pre-committed grades in Table 14. The tasks were written by an Opus 5.5 agent; their hidden tests are not published.

Later tool version C2. C2 notes come from a development version of `since-cutoff` (not released) that adds the dependency-switch notes of 0.5.0, a 30-day margin before the cutoff and a new detector for flipped boolean defaults of public parameters. Coverage was recorded before any session: pl01 (changed default) and mi01 (dependency switch) became covered; pl03 and pl04 stay below the five-line cap, st02's ignored parameter is invisible to a static diff, and the margin hides mi02's flipped default. C2 also changed wording: its st01 bullet folds `st.cache` into a list of four names, and its pl02 bullet quotes the library's text instead of naming `scan_csv`. Agents solved pl01 and mi01 without notes too, so coverage bought cost, not correctness, here. The concurrent single-repetition comparators favour C2 (C2 over concurrent C 0.83), and drift was not flagged, so the primary result is inconclusive rather than evidence that C2 is worse.

F Open-Weight Agent

Setup. The model is Qwen3-Coder-30B-A3B-Instruct (Apache-2.0; mixture of experts with 3.3B active parameters) as a 4-bit GGUF file (Q4_K_M), served by llama.cpp (build b11821) on one 16 GB laptop GPU with the experts of 20 of 48 layers in CPU memory, two parallel slots of 40,960 tokens and the model card's sampling settings (temperature 0.7, top-p 0.8, top-k 20, repetition penalty 1.05). The scaffold is mini-swe-agent 2.4.6 with its default configuration and one bash tool called natively, at most 40 model calls (the Claude runs' cap was 40 CLI turns), 1,800 s per session (never reached: longest 563 s, median 123 s) and 180 s per command. An adapter reuses the harness's tasks, pinned environments, CLAUDE.md texts and hidden-test grading; the condition's CLAUDE.md text is written to the project and also prepended to the task prompt. Agent commands ran with an environment built from scratch, offline pip and a dead proxy, but without a filesystem sandbox. The model card states no training cutoff; we used the value in the tool's model registry, April 2025 (the model was released on 31 July 2025), as a user of the tool would. Notes came from the released 0.4.1 with unchanged settings, generated for that cutoff (for example anthropic 0.50.0, openai 1.76.2 and polars 1.29.0 as cutoff-era versions) and frozen before any session. The diffs are larger than for the 2026 cutoff and 0.4.1 keeps five lines per package, so the notes named the task's change in only 6 of 31 treatment tasks (ant01, hf01, hf03, lc01, lc02, st01), which the addendum recorded before the runs.

Protocol. Addendum section 10 was committed and tagged before any GPU session, including a two-session pilot that only fixed the slot count. It re-assigned strata for the 2025 cutoff: 31 treatment tasks (the 25 Opus post-cutoff tasks plus pd01–pd03, lc01, lc02 and st90, whose changes postdate April 2025), two controls (pl90, pl91) and two descriptive tasks (oai04, oai05). A and C ran three repetitions in randomised batches, all first repetitions before any second one. The rules were: pass rate primary (Δ(C−A) CI above 0 and sign-flip $p < 0.05$), with a floor rule that forbids pass-rate claims if A and C both pass under 10%; total tokens (C/A CI below 1); stale first edit (sign-flip); controls descriptive; sensitivity analyses without pd01 and on the 25 tasks that are post-cutoff for every model. Amendment 10a, written during the A/C runs and after interim A/C numbers had been seen, and committed before any H session, added H on the 17 original post-cutoff tasks (51 sessions) with a placebo test (C better or cheaper than H) and the same floor rule. All 261 sessions are valid at the first recorded attempt (one H session interrupted before writing a record was re-run; Deviations), and every batch passed the secret scan.

Results. No rule was met (Table 15). Pass rate was 19.4% without notes and 24.7% with them, not significant; notes did not reduce tokens (C/A 1.08), unlike Opus 5.5 (0.73–0.83), and stale first edits fell from 39% to 32%, not significantly. A quarter of the A and C sessions (53 of 210) stopped after three consecutive replies without a tool call. A diagnosis after the study traced this to our adapter: its prompt templates asked for a fenced text command while its model class accepted only native tool calls, so the fault hit all arms alike but lowered every pass rate; without these sessions pass rates were 25.0% and 28.8% (exploratory). Where the notes named

the change, C produced the only passes on hf01 (2 of 3) and hf03 (1 of 3) and fewer stale first edits (7 of 12 to 3 of 12), but all three C runs of lc02 stopped on format errors before any edit (A passed 3 of 3), and tokens rose (1.37, CI includes 1). In one ant01 C run the agent restated the note that `temperature` is no longer accepted and still kept `temperature=0` in its final code. On the controls C failed 3 of 6 runs and A none: one wrong result and two format-error stops; with two tasks this is descriptive, but no C run had failed a control task in any earlier study. On the 17 original tasks the floor rule fired; H was the cheapest arm (H/A tokens 0.79, CI includes 1) and C used more tokens than H (1.42 [1.04, 2.02]), so the placebo test was not met: on this model the change lines added cost without a detectable benefit.

A second open-weight study, stopped. A second open-weight study was pre-committed (addendum section 15) after the diagnosis above and before its first session: the same model, server and tasks with an adapter that asks for native tool calls, and five arms in the same randomised batches (A, C, H, the lockfile-only header L of Appendix E, and a decoy header naming an unrelated installed package). It was started and stopped before completion; its partial runs are not analysed, and neither its protocol section nor its runs are in this report's data.

Deviations. (1) H was pre-committed after interim A/C results and ran after and partly alongside the A/C sessions, so it is not randomised with them. (2) For about two hours one H client shared the two server slots with the A/C clients, which lengthened late sessions; wall time is used for no claim, and one interrupted H session, which had written no record, was re-run. (3) Stale patterns match the idioms of the June 2026 cutoff; this model's older idioms sometimes differ (huggingface-hub 0.30 used `requests`), so its stale rates are a lower bound. (4) No filesystem sandbox; (5) one model, one quantisation and one scaffold; (6) the format-error stops came from our adapter's templates (Results). Results are descriptive relative to the Claude studies, which used another agent, other limits and another cutoff.

G Detector Recall and Precision

Design. The protocol was written before any detector ran on these pairs and before the ground truth existed, and the ground-truth files were frozen, with their hashes recorded, before the first tool run; neither was committed to git, so this is not a pre-commitment in the sense of §4. Each of the seven libraries with post-cutoff tasks is compared from its cutoff-era to its pinned release: anthropic 0.115.0 to 1.8.0, openai 2.44.0 to 3.19.2, huggingface-hub 1.21.0 to 2.0.0, mcp 1.28.1 to 2.2.0, polars 1.42.1 to 2.0.0, streamlit 1.58.0 to 1.66.1 and mistralai 2.5.0 to 3.3.0. The ground truth holds breaking or behavioural changes to the public Python API that the official release notes, changelogs and migration guides document for the releases in between, each verified against both wheels (old form present in the cutoff-era release, change present in the pinned one); a seeded random sample of up to 30 verified items per library gives 154 items (anthropic 30, openai 4, huggingface-hub 18, mcp 30, polars 30, streamlit 12, mistralai 30): 54 signature changes, 42 removals, 32 behaviour-only changes, 22 renames or moves, 3 dependency switches and 1 default change. The tools are the evaluated 0.4.1, the development version behind C2 (which adds dependency-switch and changed-default detection) on the same pair, `griffe check` 2.4.1 and AexPy 0.4.3 (a change counts when ranked Low, Medium or High). Detector recall counts every report, with no cap; notes recall counts the block of a project that only imports the package, with five lines per package (the benchmark setting) or ten.

AI judging. Five Claude Opus 5.5 sessions built and verified the ground truth in one pass, blind to all tool output. Claude Opus 5.5 then matched reports to items (a rule-based proposal, then a review of every item, with 21 item-level overrides and their reasons) and judged precision. A report is *accurate* if the stated change exists between the two releases and the old object is not still reachable through a re-export, `__getattr__`, inheritance or `**kwargs`; *breaking* if it is also breaking for code on a public path. No human reviewed any of these judgments.

Results. The three static diffs find largely the same changes (Table 16). In exact McNemar tests (exploratory, unadjusted), 0.4.1 found no item that `griffe check` missed and missed 10 that it found ($p = 0.0020$); 7 of the 10 are classes that `griffe check` reports as removed because it does not follow a wildcard re-export, while only their fields were renamed, so counting only accurate matches it finds 50.6%. The development version did not differ from `griffe check` (5 against 2 discordant

items, $p = 0.45$) or AexPy (12 against 6, $p = 0.24$). The 59 items no tool found are the 32 behaviour-only changes, 9 polars parameters removed behind deprecation decorators with identical signature text, 14 annotation-only type narrowings, 2 required fields added to response models, a name reused for a different class and a re-export dropped from a module without `__all__`. No item reached a block without being detected; the cap only removes, and its line order (broken imports, known replacements and removals first) makes the block almost removal-only. On the 25 task changes, 0.4.1 detected 12, the development version 20, `griffe check` 15 and AexPy 20; five that 0.4.1 detected (mcp01, mcp04–mcp06, pl03) fell below the cap of the frozen blocks, and no tool detected oai03 (stream errors are now wrapped) or pl04. Two block lines name a dubious replacement although the removal is real.

Deviations and caveats. Matching rules were refined after seeing outputs, uniformly for all tools, and the accurate-matches sensitivity was added post hoc. The one-file-project block differs from the frozen benchmark blocks by design, as those rank the APIs the task code uses first. mistralai is a namespace package, so its one-file project imports `mistralai.client`. One ground-truth session looked at about 12 candidates past its 30th verified item; none is used. Behaviour changes made only in compiled code are under-represented: 13 polars changes made only in Rust could not be verified and were not sampled. openai contributes only 4 items. Pooled intervals ignore clustering by library, and one model judged everything.

H Exposure Scan Details

Random two-stratum sample. The protocol was written, and its sha256 recorded, before the frame was enumerated, any repository selected or any result existed; it was not committed to git, so it is not a pre-commitment in the sense of §4. It fixed the frame, eligibility, strata, analysis and audit. The frame is every Python repository found by GitHub search with at least 200 stars, a push on or after 25 July 2026, not archived and not a fork, enumerated in 29 star and date slices on 23 October 2026: 15,624 repositories (of the 15,627 the search reported), and 15,424 after removing 77 templates and 123 repositories over 2 GB, shuffled with a fixed seed. Walking that permutation, a repository was eligible if it had a pin source at the root (a uv, Poetry, PDM, pylock or Pipfile lockfile, or a requirements file), could be sparse-cloned, was scanned successfully within 1,200 s, and took at least half of its checked direct dependencies from a lockfile, an exact pin or an environment. Of 1,809 candidates walked, 877 had no pin source at the root, 12 failed to clone on Windows (10 paths invalid there, 2 too long), 7 scans found nothing to check, none timed out, 141 had fewer than half of their direct dependencies pinned, 488 were skipped after G was full because no dependency file named a library on the AI list, and 284 were eligible. G is the first 100 eligible repositories (filled at candidate 316), AI the first 100 eligible AI-stack repositories (a direct, imported dependency on a fixed list of LLM, agent and model libraries; filled at candidate 1,809). Nineteen repositories are in both strata, so 19% of G is AI-stack [12.5, 27.8]; one repository of the union is also among the earlier 47. Without its AI-stack repositories, G's three rates are 43%, 32% and 4.9% (81 repositories, descriptive). A sparse and a full shallow clone of the same commit gave identical scans for 3 of 3 earlier repositories, and the same definitions applied to the earlier 47 reproduce their blocks exactly. Shares have Wilson intervals and an owner-cluster bootstrap (10,000 resamples), in which the repositories of one GitHub owner are resampled together (G: 98 owners, AI: 96, at most 2 repositories per owner). Excluding the tool's fallback to PyPI's latest release from "pinned" lowers the earlier 47's counts from 38 to 36 and from 31 to 30 (from 81% and 66% in Table 10 to the 77% and 64% of Table 17).

Second audit. The pool was the 253 matched APIs of pinned packages in 44 repositories of the union; 60 were drawn with a fixed seed, from 24 repositories (matches: 21 from G only, 32 from AI only, 7 from both). One repository contributed 18 matches (a package whose new release removed two parameters from every primitive), the next 6. For each match, Claude Opus 5.5 read the matched files and both releases' sources and assigned a runtime class; 17 snippets it wrote (never repository code) ran in isolated environments with only the pinned release from PyPI: imports, signature binds, one local MCP server round trip and one local websockets server (Table 18). Fourteen of them checked true uses (the 14 of Table 18 and §6) and three checked false positives. Sixteen ran cleanly; the one ImportError concerned a name imported only for type checking (`WriteFile` from

vercel.sandbox), in a match judged not breaking (a false positive). The three old-form httpx uses still work, as the SDKs accept a legacy client or only duck-type the response. Deviations: the method check compared a fresh full and a sparse clone of the same commit rather than the earlier full clone; the 12 clone failures, a platform artefact, were treated as ineligible; the sensitivity rows and the Wilson upper bound for 0/60 are exploratory; and one rule was fixed during judging (a project class that implements or subclasses the changed interface is a true positive; 2 matches).

Earlier convenience sample. Before the random sample, a pre-committed scan took public Python repositories with at least 200 stars, a push after 1 September 2026 and a uv or Poetry lockfile at the root, in descending order of stars (no code executed). Relative to the June 2026 cutoff, 38 of the 47 (81%) lock a direct dependency to a later release, 31 (66%) import a package whose public API broke after the cutoff release, and in 10 (21%) the matcher found code that uses a changed API (53 distinct APIs). An AI assistant classified all 53 matches against the code and both releases' sources, without human review, and judged 46 true positives, a precision of 87% (Wilson 95% CI 75–93%). Of 205 repositories examined, 65 had the required files (pyproject.toml and a lockfile at the root). The pre-committed rule took the first 50 repositories whose scans succeed at both cutoffs. Two failed to clone; in four the tool found no dependency it could check, at either cutoff (eight unusable scans); and at the older cutoff 12 scans hit the 30-minute limit. These sets do not overlap, which leaves 47 repositories rather than the planned 50; Table 10 gives their exposure at both cutoffs. A secondary set of the first 50 repositories whose current-cutoff scan succeeded gives 84%, 72% and 26% for the three exposure rates. Timeouts affected large AI repositories at the older cutoff, so that column understates exposure. Scan times are as logged by the scan script (version 0.5.0, four scans in parallel on one laptop, downloads included; longest 10.4 min for 483 dependencies, 465 of them direct); the warm-cache re-runs took 1.1, 2.5, 11.0, 31.7, 70.9 and 88.1 s for repositories with 8 to 166 direct dependencies.

At the 2026 cutoff the 53 matched APIs carry 128 changes. Table 11 summarises the audit; the data folder's audit file (analysis/realworld_audit.csv) lists every match with its verdict and reason. Resampling the ten repositories instead of the matches widens the precision interval to 53–100%. A true positive means that the code references the API and the reported change exists in the pinned release. The three wrong-object matches took top-level anyio names for deprecated anyio.abc aliases; the four non-breaking changes moved parameters into **kwargs or a stub attribute to a metaclass. Of the 46 true positives, 5 are matched in the old form, one is guarded by a version check and 40 are used in a form that still works at the pinned version. The 5 old-form matches are all in one repository: it imports the removed mcp.server.fastmcp module (two matches, which break), calls two subscription methods that mcp 2.3 deprecates (which only warn), and passes an httpx client where mcp now takes httpx2 objects (flagged statically, but it still works at runtime; Appendix E). Code that actually breaks is therefore 2 matches in one repository. The yt-dlp match already branches on the websockets version. Two matches resolved to a sync or async twin with the identical change, which the matcher treats as one API; we count them as true positives.

For the probe comparison, the tool's model-probing run command (the default subcommand of 0.4.1; sync, scan and status call no model) had asked two older models 40 tool-less questions each about changed APIs in a 36-library stack, of which 31 and 29 libraries had probes. The number of breaking changes correlated with the first model's probed stale-answer rate (Spearman $\rho = 0.43$, uncorrected $p = 0.015$) but not significantly with the second's ($\rho = 0.26$, $p = 0.18$); libraries had about 1.2 probes each on average, so this is weak evidence that the diff points where models fail.

I Agent-Session Estimate of f

Data and protocol. The author's Claude Code transcripts, which the agent stores locally, were analysed on the author's machine; transcript content, code and paths stayed there, and the published data hold only aggregate counts. The protocol fixed the units, exclusions, measures, estimands, intervals and sensitivity analyses. Its sha256 was recorded before any import was extracted or any share computed, after a census of the store that counted files, models and edit calls by coarse folder but read no content. It was not committed to git, so, like the exposure protocol, it is not a pre-

commitment in the sense of §4, and the result is a descriptive estimate for one developer. The unit is an agent context: a top-level session or one subagent run, each of which would load the block once (except Explore and Plan subagents; see Cost per context). Of 1,846 transcripts we excluded 232 with no model turn, 206 from the benchmark harness and its runs, and 100 from the session that ran this paper's research and its subagents (which include this analysis). That leaves 1,308 contexts (1,281 of them subagent runs) from 28 top-level sessions (27 top-level contexts; one session's top-level transcript had been excluded) between May and October 2026, 1,199 of them in September and October; 1,120 ran Opus 5.5, 108 Sonnet 4.6, 76 Fable 5.1 and 4 Sonnet 4.5 or Haiku 4.5.

Measures. A Python edit is a successful Edit, MultiEdit or Write call on a `.py` file or a NotebookEdit of a code cell (4,639 calls). Its imports are those of the edited code plus the file's import header at edit time, taken from the latest full view in the transcript (a Write, a Read from line 1 or the edit's record of the original file; 2,506 calls), else from the file on disk now (imports only; 914), else from import lines seen for the same path anywhere in the store (167). For 1,052 edits (23%), mostly in since-deleted clones of open-source repositories, no header was recovered. Python written through shell commands is not counted. Most contexts are subagent runs, and a task split across subagents counts several times. Imported modules were mapped to PyPI distributions with the local environments' metadata, the import names recorded by the exposure scans and an alias table, after dropping standard-library and project-local modules; 87 public distributions were resolved, and 68 module names (in 137 edits) were not and count as unchanged. The version in use is the latest final release on or before the session date, which overstates recency for pinned projects. A distribution is changed if `since-cutoff` 0.5.0's static `scan` reports a public API change between the release current at the model's cutoff and the version in use, the exposure definition of §6; 42 contexts import a distribution the scan could not check (two above its download size limit, one metapackage), counted as unchanged except in S3. f_{upper} is the share of contexts that edit a Python file importing a changed distribution; f_{strict} is the share whose edited code itself uses a changed API by the matcher (the header plus the edit, minus the header alone). The break-even's f (§5, Table 3) is the share that use a changed API, which f_{strict} estimates from below; f_{upper} bounds it from above and is the relevant share only if runs that merely import a changed package also save. Intervals are percentile bootstrap intervals clustered by project (170 clusters, 10,000 draws). The largest cluster, the subagents of one session that edit nothing, holds 27% of contexts, so intervals over all contexts are wide.

Results. Of the 513 Python-editing contexts, 30.2% [19.8, 37.8] imported a distribution with a post-cutoff release in use, 17.7% one whose API changed, and 2.1% used a changed API (Table 19). The changed distributions behind f_{upper} were mostly anyio (20 contexts), hypothesis (16), pillow (13) and mcp (10), of 35 in all; behind f_{strict}, mcp (6), polars (4), wandb (1) and huggingface-hub (1). f_{upper} held for 32 of 275 contexts in open-source contributions (133 repositories), 36 of 141 in the tool's own repository, 10 of 40 in another project of the author, 11 of 84 on scratch scripts, 2 of 18 elsewhere and none of the 750 contexts that edit nothing. Pooled to the 28 top-level sessions (4 with Python edits, 3 meeting f_{upper}, 2 meeting f_{strict}), there are too few units to use.

Cost per context (post hoc). A benchmark session made 11.3 model calls (the CLI reports 16.6 turns, which count tool round trips; Appendix D), but f counts contexts, 1,281 of 1,308 of them subagent runs. The Claude Code documentation on subagents (`https://code.claude.com/docs/en/sub-agents`, fetched 24 October 2026) states that Explore and Plan subagents start without CLAUDE.md and that other built-in and custom subagents load it unless a custom definition sets `omitClaudeMd`. (The page also says subagents skip the main auto memory, which these transcripts do not show, so we rely on it only for CLAUDE.md.) In these transcripts the channel that injects CLAUDE.md and memory files was present before the first model call in all 597 general-purpose, 568 of 673 custom and none of the 9 Explore contexts (no project CLAUDE.md lay on their load path, so it carried only the memory file). Table 20 counts model calls (distinct assistant messages) per context and prices the 704-token block at each mean with one cache write, with the main study's s. Pooled as f counts them, contexts average 45.3 calls (median 29; general-purpose 47.4, custom 39.9, Explore 50.1), so c = \$0.0120 and f^* = 13.8% at $u = 0$, and the 8.3–

28.2% range of real block sizes and savings at benchmark length becomes 13.8–49.4%. One 1-hour cache write per context is an approximation: idle gaps and compactions add writes, while most writes here used the cheaper 5-minute cache, so c may be somewhat higher or lower. s comes from single-context benchmark sessions of 11.3 model calls, and whether the saving grows with context length is untested; K comes from these contexts' own logged usage ($2.96 per context on average, $5.97 for Python-editing contexts; Appendix D), so c/K is 0.40% and 0.29%. Counting every context of a top-level session (46.7 on average), the block would cost $0.56 per session, 6.4 times a single-context saving. With 27 main-session contexts, of which 3 edit Python, f cannot be estimated per main session.

Against the break-even. At the contexts' own length (Table 20), f_{upper} lies below the $u = 0$ break-even both among Python-editing contexts (17.7% against 19.6%) and over all contexts (7.0% [2.4, 16.2] against 13.8%), though both intervals reach above it; imputing the missing headers raises the two shares to about 29% and 11%, the first above its break-even (19.7% in the unified bootstrap, where at $u = 0$ the block pays in 95% of draws, an overstatement because the imputed rates are held fixed). f_{strict} is far below every break-even. It is a lower bound, because the matcher's recall is below 1 (Appendix G), 608 of the 1,884 scanned edit snippets (32%) did not parse as Python and could not be matched, and headers are missing; even so, the upper end of its Python-editing interval (3.4%) is well below 19.6%, its break-even at those contexts' length (8.3% even at benchmark length). The benchmark measured the saving on tasks whose cutoff-era solution fails, a narrower case than f_{strict} counts (most real changed-API uses still work, §6), so a context counted by f_{strict} likely saves less than s. For this developer, an always-on imported-scope block would therefore repay its cost through affected contexts alone only at the imputed Python-editing share; otherwise it pays only if the other contexts, including those that import a changed package without using a changed API, get cheaper, by at least 0.38% at f_{strict} and 0.22% at f_{upper} for the 704-token block over all contexts (K measured; Appendix D). Our control tasks neither show nor exclude that (point estimate 6.6%, interval including zero). Leaving the block off is the loss-averse course; a live A/B test of the net effect decides.

Deviations. All were decided before f was computed unless marked post hoc. (D1) The module-to-distribution map also uses the import names recorded in the exposure scans' reports, because the local environments cover few third-party packages. (D2) Subagent transcripts store no structured tool results, so headers were read from the numbered Read output, and the last fallback (import lines seen for the same path anywhere in the store, including partial reads) was added. (D3, post hoc, after 23% of edits were found to lack a header) S6 and S7. (D4) Errored tool calls are not edits; generic local names such as `tests`, `src` and `utils` are never third-party; scans were grouped by cutoff and date, which is equivalent because a package's status depends only on its versions and the cutoff. Also post hoc: the breaking-only row and the shares among contexts that edit any file (16.3% [9.9, 20.6] for f_{upper}, 2.0% [0.3, 3.1] for f_{strict}).

J Reproducibility

The report's folder in the tool's repository, `research/dependency-block-study`, holds this paper's source (`paper/`), the data (`data/`) and the scripts (`scripts/`); its README lists the commands. The data contain the protocol and its addendum (including the sections that pre-committed the new tasks, the four follow-up studies, the concurrent study, the open-weight agent with its amendment, and section 14 with its frozen L and W blocks and analysis script, registered but not run; section 15, a second open-weight study that was stopped before completion, is not included), the frozen notes for every model, task set and condition (H, R, G, C2, the older-cutoff, case-study and open-weight notes), the configuration files and the open-weight agent's server, adapter and batch scripts, the definitions of all 35 benchmark tasks and the two case-study tasks, the review report for the new tasks, the analysis scripts (token types, exact tests, block size and both break-even models, header break-even, size cap and sample sizes, the post hoc coverage split and equivalence tests, the analysis without the vendor's SDK, and the Table 1 and Figure 1 numbers), every aggregated result file, the audit file, and de-identified per-run tables of all eleven completed studies (task, condition, repetition, pass, tokens by type, turns, wall time, stale-edit flags, exploration counts and, for the open-weight agent, exit status), from which the scripts reproduce every

ratio, interval and test; `data/README.md` names the script and output file behind each appendix analysis. The Context7 counts of Appendix D come from a script over the public main-study transcripts, whose output lists the sessions behind each count. Repositories and hidden tests of the new and case-study tasks are not published, so that the new tasks stay unseen. Costs use the list prices of Appendix K, checked against Anthropic's pricing page (`https://platform.claude.com/docs/en/about-claude/pricing`) on 24 October 2026. For the detector evaluation and the random exposure sample the data contain both protocols, all scripts, the ground-truth items with their sources and verification evidence, the match adjudications and precision verdicts, and the aggregated results; the raw scans, blocks and audit verdicts of the random sample name repositories and are not included. For the agent-session estimate of f they contain the protocol, the analysis script and the aggregate outputs (including the per-context counts and the unified break-even); the transcripts are private and not included, so these scripts cannot be re-run from the published data. They also list, for each study, the time of its pre-commitment tag and of its first session; every tag precedes the study's first session. The main study's protocol is tagged `main-freeze-2026-09-28` (commit de23b80) in the public benchmark repository; the addendum tags are in a local branch that has not been published, so their times cannot yet be checked externally. Agent runs used Python 3.13, wheels installed offline from hash-locked environments, and fixed seeds for the run order and the bootstrap. Deviations recorded in the protocol: condition E added after the pilot; notes generated with the released tool version 0.4.1 instead of the planned 0.4.0 release candidate; the GLMM robustness check was not run; sessions used the maintainer's normal login rather than an isolated home directory, which is how the token in the Ethical Considerations became readable; the exposure scan yielded 47 instead of 50 repositories; and one main-study control session (pd02, None, repetition 1) was re-run, as the protocol prescribes for infrastructure errors, after its grading environment could not be created (the second attempt is the valid one). The addendum's price table for the smaller models gave Sonnet 5.5's cache reads as $0.20 per million tokens; we use the list price, $0.10, which changes only Sonnet cost figures, none of them a pre-committed metric. The new-task study has no recorded deviation. In the follow-up studies, a concurrent, additive harness change that defined C2 took effect during the older-cutoff runs and before the case study (conditions A, C and R unchanged), and the case study's flaky hidden test led to the post hoc regrade in Appendix E. The concurrent study has no deviation beyond a wrong clock time in its addendum heading (the commit precedes the first session). The open-weight study's deviations are listed in Appendix F. The main study's transcripts are public; those of the later studies are not published, and the data hold de-identified per-run tables instead.

Infrastructure checks. Three problems raised no error in the logs: the Context7 quota error came back as a normal tool result, Claude Code priced the smaller models at a fallback rate, and early sessions could read a maintainer's token (Ethical Considerations). We found them in transcripts and a secret scan, recomputed cost from token counts and isolated credentials.

K Threats to Validity: Details

This appendix continues the Limitations section; it holds detail, not further results.

Task provenance. The 24 original tasks were written before any study by one design session and five per-library sessions, and the 11 new tasks after the first results, in libraries chosen with the tool's own diff and runtime probes. Separate Opus 5.5 sessions reviewed each set: one removed old-API code from seven original task repositories and added alternative solutions to nine tasks, and the other ran 55 alternative and wrong solutions against the new tasks' hidden tests (Appendix A). Opus 5.5 solved all new tasks without notes and wrote no stale first edit there, so that set tests cost only, and the concurrent five-arm study also ran on it. Several new-task changes were announced by deprecation warnings before the cutoff.

Open-weight agent. Qwen3-Coder-30B-A3B ran with one 4-bit quantisation and one scaffold on a local GPU. A quarter of its sessions ended in format errors caused by our adapter's prompt templates, the notes named the task's change in 6 of 31 tasks, stale patterns written for newer idioms give a lower bound, H was added after interim A/C results and not randomised with A and C, and agent commands had no filesystem sandbox (Appendix F).

Cutoff anchoring. Notes anchored to the older cutoff of Sonnet 4.5 lowered that model's pass rate from 82% to 71% (CI includes 0) on 15 tasks and covered only 2 of them. Whether anchoring to the model's cutoff, rather than to the project's previous pin or a fixed window, is what makes the block useful remains untested.

Task ecology. Dependency-upgrade benchmarks for coding agents (Luo et al., 2026) could test the saving on larger repositories. The break-even share assumes that the saving per affected session measured with 423-token blocks carries over to real blocks about twice that size and to contexts about four times as long; sessions that reuse a warm cache within the hour pay only cache reads, which would lower the overhead.

Controls for content. G and H differ in more than version specificity: G prescribes a lookup procedure, while H names the packages and claims authority. R's header still says the lines concern the project's own packages, and its donor packages are not installed. No equivalence margin was pre-committed for C against H; a post hoc $\pm 10\%$ test on cost fails in the pools, as does one at the $\pm 15\%$ margin our protocol pre-committed for the related H/L comparison (Appendix D). Only 7 of 25 blocks named the task's change, so the content was mostly untested, and the covered-task contrast is post hoc. The earlier, non-concurrent R result (C/R 0.78) is superseded and kept only in Appendix E. The exploration analysis was not pre-committed. On our single-library tasks the pre-committed lockfile-only header L would name the same package as H, so it can test the cutoff framing but not the diff's selection.

Coverage. The frozen notes covered the task's change in 5 of 17 original and 2 of 8 new post-cutoff tasks (C2: 4 of 8; open-weight model: 6 of 31); 0.4.1 lists at most five changes per package and missed a dependency switch behind five tasks (ant04, hf04, oai01, oai02, mi01). Against documented changes, the diff found about half and the five-line block about one in ten (AI-judged ground truth and matching; Appendix G).

Case study and later version. The case study has two tasks in one repository, written by an Opus 5.5 agent; its pre-committed grades were confounded by a flaky hidden test, and the corrected-test grades are post hoc. The changed-defaults detector in C2 was written after the evaluation showed pl01 uncovered, and C2 differs from C in wording and comparison release as well as coverage.

Pilot exposure. The tool was changed after seeing ant01 in the pilot; we report all results without ant01 and mcp01, and some effects shrink.

Efficiency measures. The pre-committed efficiency rule changed from a four-metric disjunction that includes cost (main study) to total tokens alone (later studies, because the CLI mispriced the smaller models). List-price cost per solved task, computed after the studies, is descriptive only (Weinberger and Hozez, 2026): it adds C's extra passes, which the ceiling rule bars as a claim, to the cost ratio (main study 0.80 to 0.76, from ant01), and on Sonnet 5.5 it shows a saving (0.67) that per-session cost (0.94) does not. Per-session ratios include failed runs, so on the smaller models they mix efficiency with success. Percentile bootstrap intervals over 8–25 tasks can be too narrow, so claims rest on the exact sign-flip test.

Pricing. All costs are recomputed from logged token counts at Anthropic's list prices per million tokens: Opus 5.5 \$4 input, \$20 output, \$0.20 cache read, \$8 one-hour cache write; Sonnet 5.5 \$2, \$10, \$0.10, \$4; Sonnet 4.5 \$3, \$15, \$0.30, \$6; Haiku 5.5 \$0.10, \$0.50, \$0.01, \$0.20 (its tier for prompts up to 100k tokens; no logged prompt of the smaller models exceeded 70k). Cache reads cost 5% of input on Opus 5.5 and Sonnet 5.5 and 10% on the other two, so cost ratios weight cache reads differently across models. Tokens are the primary metric for the smaller models.

Drift. The studies ran up to 25 days apart, so between-study comparisons are descriptive. None runs of the same tasks differed in tokens by 18% over 3.5 hours (11 tasks) and 15% over 6.5 hours (8 post-cutoff tasks; 10% on all 11 tasks; Appendix E); these CIs include 1, but the identical Irrelevant block used 9% fewer tokens in the five-arm sitting than in its own study hours earlier (0.91 [0.82, 0.99]; Table 14), so drift as large as the effect cannot be excluded and was detected twice by the bootstrap (that block's tokens; None's cost over 3.5 hours, 1.17 [1.002, 1.35]). The main study and its replication found their saving on different tasks (per-task log cost ratios $r = -0.19$; Appendix D), and our per-study intervals have no run-day component; the random-effects estimate over the four Opus studies (Appendix D) found no

detectable between-study variance in the cost ratio, but it rests on four studies that share tasks.

Author role. Five of the 17 original post-cutoff tasks use the evaluated vendor's SDK; without them the main study and both pools still save while the replication does not (an exploratory subset). The main study's freeze tag is in the public benchmark repository, but the addendum tags are in an unpublished local branch, so the order of the later steps cannot yet be checked externally; transcripts of the later studies are not published.

task	library (pinned; cutoff era)	stratum	what the cutoff-era code gets wrong
ant01	anthropic 1.8.0; 0.115.0	post-cutoff	`temperature=` on `messages.create` raises `TypeError`
ant02	anthropic 1.8.0; 0.115.0	post-cutoff	raw-response accessor now returns a bound method (silent)
ant03	anthropic 1.8.0; 0.115.0	post-cutoff	beta structured-output keyword rejected
ant04	anthropic 1.8.0; 0.115.0	post-cutoff	`httpx.Client` as `http_client` rejected (now `httpx2`)
ant05	anthropic 1.8.0; 0.115.0	post-cutoff	async `parse()` is not awaited (coroutine)
hf01	huggingface-hub 2.0.0; 1.21.0	post-cutoff	`stop_sequences=` removed
hf02	huggingface-hub 2.0.0; 1.21.0	post-cutoff	`upload_large_folder` removed
hf03	huggingface-hub 2.0.0; 1.21.0	post-cutoff	`list_models(model_name=)` removed
hf04	huggingface-hub 2.0.0; 1.21.0	post-cutoff	`import httpx` fails (dependency switched)
mcp01	mcp 2.2.0; 1.28.1	post-cutoff	`FastMCP` import path and error handling changed
mcp02	mcp 2.2.0; 1.28.1	post-cutoff	tool-result fields renamed
mcp04	mcp 2.2.0; 1.28.1	post-cutoff	host/port no longer constructor arguments
mcp05	mcp 2.2.0; 1.28.1	post-cutoff	streamable-HTTP client renamed
mcp06	mcp 2.2.0; 1.28.1	post-cutoff	in-memory client helper removed
oai01	openai 3.19.2; 2.44.0	post-cutoff	`import httpx` fails (dependency switched)
oai02	openai 3.19.2; 2.44.0	post-cutoff	test double patches the old transport (silent)
oai03	openai 3.19.2; 2.44.0	post-cutoff	transport errors from the old HTTP library
pd01–pd03	pandas 3.0.6; 3.0.3	pre-cutoff control	string dtype, `applymap`, `fillna(method=)`
lc01, lc02	langchain-core 1.6.5; 1.4.8	pre-cutoff control	retriever and chat-model calls removed earlier
oai04	openai 3.19.2; 2.44.0	no-change control	everything needed works in both versions
oai05	openai 2.44.0; 2.44.0	pinned behind	the 3.x answer imports a library 2.44.0 lacks

Table 4: The 24 original tasks. Notes covered the change for ant01, ant03, hf01, hf02 and hf03 (and partly mcp01, counted as uncovered in the coverage split; Appendix D reports both). Blocks were empty for oai05 and pd01–pd03, where C equals A.

task	library (pinned; old)	stratum	what the old code gets wrong
pl01	polars 2.0.0; 1.42.1	post-cutoff, silent	`explode` default `empty_as_null` now False: rows with empty lists disappear
pl02	polars 2.0.0; 1.42.1	post-cutoff	`read_csv_batched` removed
pl03	polars 2.0.0; 1.42.1	post-cutoff	`Expr.replace(default=)` removed (`replace_strict`)
pl04	polars 2.0.0; 1.42.1	post-cutoff	`collect(streaming=True)` removed (`engine="streaming"`)
st01	streamlit 1.66.1; 1.58.0	post-cutoff	`st.cache` removed
st02	streamlit 1.66.1; 1.58.0	post-cutoff, silent	`st.image(use_column_width=)` accepted but ignored
mi01	mistralai 3.3.0; 2.5.0	post-cutoff	custom HTTP client: `httpx` no longer installed (now `httpx2`)
mi02	mistralai 3.3.0; 2.5.0	post-cutoff, silent	`ocr.process` default `include_blocks` now True
pl90	polars 2.0.0; 0.20.31	pre-cutoff control	`cumsum` removed in 1.0.0
pl91	polars 2.0.0; 0.20.31	pre-cutoff control	`DataFrame.groupby` removed in 1.0.0
st90	streamlit 1.66.1; 1.53.1	pre-cutoff control	`experimental_get/set_query_params` removed in 1.54.0

Table 5: The 11 new tasks (private; written after the first results were known). The old version is the cutoff-era release for post-cutoff tasks and, for controls, the release the stale code was written for. Notes covered the change for pl02 and st01. Every block was non-empty, including the controls.

tag	what was checked; example note
[diff]	static API diff only; "anthropic.AI_PROMPT was removed; do not use it."
[diff + library]	the library's deprecation text names a replacement that exists in the pinned version; "Use HfApi.duplicate_repo instead." (also Figure 1)
[diff + move checked]	the object at the new path keeps the old public names or parameters; "import it with from anthropic.types import deleted_file."
[diff + metadata]*	the release's requirements and public signatures switched to another library; "openai 3.19.2 requires httpx2 instead of httpx."
[diff; probable rename]	a removed parameter's successor at the same position with the same type, labelled a guess; "x was probably renamed to y (same position and type)." None in our blocks
[type-checked]	an example that basedpyright accepts for the pinned version; defined in the header's tag legend, but no note in our frozen 0.4.1 blocks carries it

Table 6: Evidence tags on notes. Only [type-checked] notes are written by a model (in the tool's model-probing run command); none is in our blocks. With the opt-in --suggestions, names that merely look similar are listed as [not confirmed], never as replacements; no block of ours used it. *Not in 0.4.1: added in 0.5.0 (exposure scan, block sizes) and in C2's development version; of the agent-study blocks, only C2 carries it.

quantity	all tasks	w/o pilot
Main study, Opus 5.5 (360 sessions; A–E, 72 runs each)		
A / C pass rate	94.1% (48/51) / 100% (51/51)	100% / 100%
Δ(C−A), points	+5.9 [0.0, +17.6]; sign-flip p = 1.0	+0.0
C/A cost	0.80 [0.70, 0.90]	0.78 [0.68, 0.88]
C/A total tokens	0.73 [0.59, 0.87]	0.69 [0.56, 0.84]
C/A turns / wall time / output tokens	0.83 [0.74, 0.91] / 0.87 [0.79, 0.95] / 0.82 [0.73, 0.91]	0.80 / 0.86 / 0.79
cost per solved task, A / C	\$0.45 / \$0.34 (0.76 [0.63, 0.86])	\$0.42 / \$0.34 (0.80 [0.73, 0.88])
C/A cost, exact sign-flip	15 of 17 tasks below 1, p = 0.00027	p = 0.00024
C/A cost, covered tasks (5)	0.75 [0.52, 0.99]	
C/A cost, uncovered tasks (12)	0.83 [0.75, 0.91]	
C/A cost, controls (7)	0.97 [0.86, 1.09]	
stale first edit, A / C	11.8% / 2.0% (6/51, 1/51)	6.7% / 2.2%
tests	Fisher p = 0.11; sign-flip p = 0.38	Fisher 0.62; sign-flip 0.75
stale in final diff, A / C	13.7% / 2.0%	8.9% / 2.2%
E/A cost; C/E cost	0.85 [0.71, 0.98]; 0.95 [0.78, 1.19]	n/a; 0.94 [0.75, 1.22]
Context7 call rate in B (post-cutoff)	19.6%	15.6%
total list spend A / B / C / D / E	\$24.92 / \$24.58 / \$20.59 / \$20.31 / \$22.57	
Sonnet 5.5 (216 sessions; A, C, H)		
A / C / H pass rate	74.5% / 90.2% / 90.2%	84.4% / 91.1% / 95.6%
Δ(C−A), points	+15.7 [+2.0, +31.4]; p = 0.094, Holm 0.19	+6.7; p = 0.38, Holm 0.75
Δ(H−A), points	+15.7 [+3.9, +31.4]; p = 0.062, Holm 0.19	+11.1; p = 0.13, Holm 0.38
C/A tokens / cost / turns	0.93 [0.75, 1.14] / 0.94 [0.76, 1.13] / 1.00 [0.90, 1.13]; smallest token ratio detectable with 80% power about 0.74	0.91 / 0.93 / 0.98
cost per solved task, A / C / H	\$0.13 / \$0.09 / \$0.11; C/A 0.67 [0.47, 0.89], H/A 0.85 [0.69, 0.99]; summed cost C/A 0.81	C/A 0.76 [0.57, 0.95]
Opus main C/A ÷ Sonnet C/A, tokens (post hoc)	0.78 [0.58, 1.04], sign-flip on log ratios p = 0.13	
H/A tokens	1.13 [0.96, 1.35]	1.09
C/H tokens / turns	0.83 [0.67, 1.01] / 0.88 [0.77, 0.99]	0.84 [0.67, 1.04] / 0.89 [0.77, 1.02]
stale first edit, A / C / H	33.3% / 28.0% / 19.6%; C vs A p = 0.67	24.4% / 27.3% / 15.6%
Haiku 5.5 (144 sessions; A, C)		
A / C pass rate	80.4% / 86.3%	84.4% / 84.4%
Δ(C−A), points	+5.9 [−7.8, +23.5]; p = 0.75	0.0
C/A tokens / cost / turns	0.92 [0.82, 1.05] / 0.99 [0.86, 1.19] / 0.96 [0.87, 1.04]; smallest detectable token ratio about 0.83	0.96 / 1.03 / 0.98
Opus main C/A ÷ Haiku C/A, tokens (post hoc)	0.79 [0.62, 0.99], sign-flip on log ratios p = 0.070	
stale first edit, A / C	25.5% / 11.8%; sign-flip p = 0.13	22.2% / 13.3%; p = 0.25
Replication, Opus 5.5 (153 sessions; A, C, H; post-cutoff only)		
A / C / H pass rate	92.2% / 100% / 100%	ceiling
C/A tokens / cost	0.83 [0.68, 0.99] / 0.85 [0.69, 1.01]; sign-flip p = 0.074 / 0.13	0.87 [0.72, 1.03] / 0.92 [0.78, 1.04]
cost per solved task, A / C	\$0.39 / \$0.29 (0.76 [0.60, 0.89])	0.82 [0.73, 0.91]
H/A tokens / cost	0.95 [0.79, 1.14] / 0.99 [0.84, 1.16]	0.90 / 0.97
C/H cost	0.86 [0.68, 1.03]	0.94
stale first edit, A / C / H	13.7% / 0.0% / 5.9%; C vs A sign-flip p = 0.13	
Pooled stale first edit, C vs A (sign-flip over 17 tasks; 15 w/o pilot)		
three models (pre-committed)	mean 9.8 points, 8 tasks differ, p = 0.055	3.7 points, p = 0.19
four original studies (post hoc, after the pre-committed pool)	mean 10.8 points, 9 tasks differ, p = 0.027	5.0 points, 7 tasks differ, p = 0.094

Table 7: Full statistics of the four original studies. Pass-rate differences are in percentage points; p is the exact sign-flip test; Holm-adjusted values are within each study's pre-committed family (three contrasts for Sonnet, four for the main study). Minimum detectable ratios are $\exp(-2.80\,\mathrm{SE})$ from the observed task-level standard error of the log ratio.

quantity	all tasks	w/o pilot
New tasks, Opus 5.5 (99 sessions; A, C, H; 8 post-cutoff tasks, 3 controls; no pilot tasks)		
A / C / H pass rate	100% (24/24) / 95.8% (23/24) / 100%	ceiling
Δ(C−A), points	-4.2 [$-12.5, 0.0$]; $p = 1.0$, Holm 1.0	
C/A tokens / cost	0.79 [0.68, 0.92] / 0.83 [0.77, 0.91]	
C/A turns / wall time / output tokens	0.77 [0.70, 0.84] / 0.91 [0.85, 0.99] / 0.80 [0.73, 0.86]	
H/A tokens / cost / turns	0.86 [0.74, 1.01] / 0.94 [0.80, 1.11] / 0.85 [0.78, 0.93]	
C/H tokens / cost / turns	0.92 [0.78, 1.07] / 0.89 [0.78, 0.996] / 0.90 [0.83, 0.97]	
C/A tokens, covered (2) / uncovered (6)	0.63 and 0.55 (pl02, st01; no CI for two tasks) / 0.88 [0.79, 0.98]	
H/A tokens, covered / uncovered	0.72 (two tasks) / 0.91 [0.79, 1.08]	
cost per solved task, A / C / H	\$0.19 / \$0.16 / \$0.18	
stale first edit, A / C / H	0/24 / 0/24 / 1/24	
controls (3): C−A; C/A tokens	+0.0 (9/9 each; bootstrap bound degenerate, exact bound for C 71.7%); 0.88 (CI not interpretable for three tasks)	
Pooled original + new, Opus 5.5 (25 post-cutoff tasks; 23 w/o pilot; 10 controls)		
main + new: A / C pass	72/75 / 74/75; Δ +2.7 [$-4.0, +12.0$], $p = 1.0$	69/69 / 68/69
main + new: C/A tokens / cost	0.75 [0.65, 0.85] / 0.81 [0.74, 0.88]	0.72 [0.62, 0.83] / 0.80 [0.72, 0.87]
replication + new: A / C / H pass	71/75 / 74/75 / 75/75; Δ(C−A) +4.0 [$-2.7, +13.3$], $p = 0.75$	68/69 / 68/69
replication + new: C/A tokens / cost	0.82 [0.71, 0.93] / 0.85 [0.73, 0.95]	0.85 [0.74, 0.96] / 0.89 [0.79, 0.97]
replication + new: H/A tokens; C/H cost	0.92 [0.80, 1.05]; 0.87 [0.74, 0.99]	0.89 [0.78, 0.9995]; 0.92 [0.84, 1.01]
combined Opus (post hoc): C/A tokens / cost	0.78 [0.71, 0.86] / 0.83 [0.77, 0.89] (each original task's log ratio averaged over main and replication, plus the 8 new tasks)	
stale first edit, C vs A (sign-flip)	main + new $p = 0.38$; replication + new $p = 0.13$	$p = 0.75$; $p = 0.25$
controls, main + new: C−A; C/A tokens / cost	0.0 (30/30 each; exact bound for C 90.5%, 84.7% on the 18 runs with a non-empty block); 0.92 [0.79, 1.05] / 0.93 [0.84, 1.04] (6 non-empty: 0.91 / 0.92 [0.82, 1.03])	

Table 8: Full statistics of the new-task study and the pooled analyses (secondary, pre-committed). Each pooled task comes from one study. Coverage split and pooled tests are as defined before the runs, except the post hoc combined estimate; p is the exact sign-flip test (Holm over C−A, H−A and C−H for the new tasks).

	package-source reads per session			task-level ratio (95% CI), sign-flip p		
study	A	C	H	C/A	H/A	C/H
Opus, main	5.94	4.22		0.71 [0.57, 0.89], 0.018		
Opus, repl.	5.84	4.53	5.37	0.78 [0.59, 0.99], 0.084	0.92 [0.74, 1.20], 0.54	0.84 [0.68, 1.02], 0.12
Sonnet	2.67	2.22	3.61	0.83 [0.64, 1.07], 0.21	1.35 [1.00, 2.02], 0.092	0.61 [0.41, 0.86], 0.016
Haiku	5.31	5.41		1.02 [0.85, 1.24], 0.88		

Introspection commands per session (for example `inspect.signature` or `help()`), A / C / H: main 0.78 / 1.41; replication 0.73 / 1.10 / 1.47 (H/A 2.03 [1.40, 3.21]); Sonnet 1.04 / 1.12 / 1.45; Haiku 1.00 / 0.80.
Reads plus introspection, ratio: main C/A 0.84 [0.71, 0.997]; Sonnet H/A 1.37 [1.11, 1.86] and C/H 0.66 [0.50, 0.84]; replication C/H 0.82 [0.68, 0.97].

Table 9: Exploration by condition (exploratory, not pre-committed; post-cutoff tasks, 51 runs per condition). Reads are Read, Grep or Glob calls on `.venv` or `site-packages`, as defined in the harness before the studies. Ratios are of task-averaged means; p-values are unadjusted, and none of the 24 all-task contrasts (four studies, three measures) survives Holm correction (smallest adjusted $p = 0.065$). The sum of reads and introspection was formed in this exploratory analysis. Without the pilot tasks: main C/A 0.69 [0.54, 0.87], $p = 0.016$; replication C/A 0.78 [0.59, 0.99], $p = 0.086$; Sonnet C/H 0.65 [0.44, 0.90], $p = 0.032$; Haiku C/A 1.09 [0.96, 1.31].

cutoff date	2026-06-30	2025-07-31
direct deps	38 [23, 75]	38 [23, 75]
share locked after	15.8% [5.8, 50.0]	76.6% [67.6, 85.3]
$\geq$1 dep after cutoff	38/47 (81%)	46/47 (98%)
imports a break	31/47 (66%)	46/47 (98%)
breaks in imports	20 [0, 92]	305 [50, 1151]
matcher finds a use	10/47 (21%)	33/47 (70%)
changed APIs used	53	212
scan time, s	26 [8, 58]	25 [12, 78]
non-empty block[a]	31/47	n/a
block tokens[a]	764 [574, 1329]	n/a
packages in block[a]	4 [2, 6]	n/a
change lines[a]	14 [8, 26]	n/a

Table 10: Exposure of the earlier convenience sample of 47 top-starred repositories (median [IQR] or count; earlier definitions, with the PyPI-latest fallback counted as pinned; Appendix H). [a]Block that version 0.5.0 would write with the benchmark's settings (imported scope, five lines per package); sizes over the 31 non-empty blocks.

repository	package	APIs	TP	FP	old form
yt-dlp	websockets	2	2	0	0
litellm	mcp	13	13	0	5
private-gpt	mcp	1	1	0	0
airflow	4 packages	7	4	3	0
paperless-ngx	django-stubs	1	0	1	0
OpenViking	openviking-sdk	22	22	0	0
posthog	temporalio	1	1	0	0
poetry	build	1	1	0	0
cognee	anthropic	2	2	0	0
openai-python	anyio	3	0	3	0
total		53	46	7	5

Table 11: First AI-assisted audit: all changed-API matches of the earlier 47 repositories at the 2026 cutoff (repository names without owners). TP: the code references the API and the change exists in the pinned release. Old form: TP matched in the form written for the old release (2 break, 2 warn, 1 still runs; Appendix H).

study (model; sessions)	pre-committed efficiency rule	outcome	result (post-cutoff tasks, ratio [95% CI])
Main (Opus 5.5; 360)	any of failing test runs, turns, wall time or cost has C/A CI below 1 (none primary)	met	cost 0.80 [0.70, 0.90], below 1 also at the Bonferroni level; turns 0.83, wall time 0.87
Replication (Opus 5.5; 153)	C/A token and cost CIs both below 1	not met	tokens 0.83 [0.68, 0.99]; cost 0.85 [0.69, 1.01]
New tasks (Opus 5.5; 99)	C/A token CI below 1	met	0.79 [0.68, 0.92]
Concurrent (Opus 5.5; 165)	C/G, C/R, G/H tokens, Holm	not met	C/G 0.80 [0.66, 0.98], Holm $p = 0.25$; C/R 0.93; G/H 1.29
Sonnet 5.5 (216)	C/A token CI below 1	not met	0.93 [0.75, 1.14]
Haiku 5.5 (144)	C/A token CI below 1	not met	0.92 [0.82, 1.05]
Qwen3-Coder (261)	C/A token CI below 1	not met	1.08 [0.87, 1.32]
Older cutoff (Sonnet 4.5; 90)	C/A token CI below 1	not met	0.88 [0.66, 1.14]
Irrelevant R (Opus 5.5; 44)	C/R token CI below 1	met, withdrawn	0.78 [0.64, 0.95]; same-batch rerun 0.93 [0.80, 1.10]
C2 (Opus 5.5; 55)	C2/C token CI upper bound at most 1.10	not met	1.03 [0.88, 1.22]

Table 12: The ten pre-committed efficiency rules, scored as written (the case study was descriptive). Two were met and still hold; R's was withdrawn after the concurrent study (Lesson 1). Pass-rate, non-inferiority, stale-edit and header-only rules are reported in the text below.

base	f	$u = 0$	u = 6.6%
all	f_{strict} 0.8%	−11 [−15, −9], 0.00	184 [−117, 590], 0.89
all	f_{upper} 7.0%	−6 [−9, −1], 0.02	177 [−104, 538], 0.89
all	imputed 11.3%	−2 [−6, 8], 0.28	172 [−94, 502], 0.90
Python	f_{strict} 2.1%	−15 [−18, −12], 0.00	372 [−217, 1,002], 0.89
Python	f_{upper} 17.7%	−2 [−8, 5], 0.27	324 [−172, 849], 0.90
Python	imputed 28.7%	8 [−2, 19], 0.95	290 [−138, 739], 0.91

Table 13: Net saving N per 1,000 agent contexts (dollars, 95% bootstrap interval, then the share of draws with $N > 0$) for the 704-token block, in one unit (model calls), with K measured in the same contexts and a fixed saving per affected use (post hoc). u = 6.6%: the estimate from the ten control tasks. Python: Python-editing contexts. Imputed: f_{upper} with missing import headers imputed (interval too narrow). With the six non-empty controls (u = 7.9%) the shares in the last column are 0.90–0.92.

quantity	value (tokens / list cost / turns, ratio [95% CI])
Concurrent five-arm study, Opus 5.5 (165 sessions; A, C, H, R, G in the same batches; 8 post-cutoff tasks, 3 controls)	
pass rate A / C / H / R / G	24/24 / 24/24 / 24/24 / 23/24 / 24/24 (ceiling); controls 9/9 / 9/9 / 9/9 / 8/9 / 9/9
C/A; H/A	0.75 [0.62, 0.90] / 0.79 [0.70, 0.90] / 0.80 [0.71, 0.89]; 0.73 [0.60, 0.85] / 0.80 [0.69, 0.92] / 0.77 [0.67, 0.89]
R/A; G/A	0.81 [0.68, 0.98] / 0.86 [0.75, 1.02] / 0.81 [0.72, 0.93]; 0.94 [0.70, 1.24] / 1.00 [0.82, 1.24] / 0.94 [0.81, 1.09]
family (tokens; sign-flip p, Holm)	C/G 0.80 [0.66, 0.98], 0.13, 0.25 (CI below 1, test not significant); C/R 0.93 [0.80, 1.10], 0.45, 0.45; G/H 1.29 [1.07, 1.59], 0.055, 0.16
C/H; covered (2) / uncovered (6)	1.04 [0.90, 1.17] (tokens); covered C/G 0.56, C/R 0.74, C/H 0.89, C/A 0.66; uncovered C/G 0.91, C/R 1.00, C/H 1.09, C/A 0.79
controls (3), tokens vs A: C / H / R / G	0.88 / 0.93 / 0.97 / 1.55 (descriptive; with three tasks the smallest attainable sign-flip p is 0.25)
drift: A / A of new-task study (6.5 h earlier); R / R of the R study	1.15 [0.91, 1.52]; 0.91 [0.82, 0.99] (tokens)
list cost per post-cutoff session A / C / H / R / G	$0.216 / $0.167 / $0.174 / $0.174 / $0.203
Irrelevant-content control R, Opus 5.5 (44 sessions; 8 post-cutoff tasks; A, C, H from the new-task study; superseded)	
pass rate R / A / C / H	100% (24/24) / 100% / 95.8% / 100%; controls R 9/9 (ceiling)
R/A	1.01 [0.87, 1.22] / 1.01 [0.87, 1.20] / 0.89 [0.80, 0.98]
C/R (pre-committed primary)	0.78 [0.64, 0.95] / 0.82 [0.69, 0.98] / 0.86 [0.79, 0.94]
R/H	1.18 [1.05, 1.32] / 1.08 [0.95, 1.23] / 1.04 [0.95, 1.14]
C/R, uncovered (6) / covered (2)	0.84 [0.69, 1.03] / 0.64 [0.51, 0.79] (tokens)
drift: concurrent A / earlier A (11 tasks)	1.18 [0.97, 1.43] / 1.17 [1.002, 1.35] / 1.05 [0.91, 1.20]; R/concurrent A tokens 0.89 [0.67, 1.16]
package-source reads per session	A 2.50, H 2.88, R 2.75, C 1.75 (exploratory)
Older-cutoff model, Sonnet 4.5 (90 sessions; A, C; 15 post-cutoff tasks; notes anchored to 2025-07-31)	
pass rate A / C	82.2% (37/45) / 71.1% (32/45); Δ −11.1 [−28.9, +4.4] points, sign-flip $p = 0.38$
C/A	0.88 [0.66, 1.14] / 0.89 [0.71, 1.08] / 0.94 [0.75, 1.13]
C/A tokens, covered (2) / uncovered (13)	1.35 (descriptive) / 0.82 [0.61, 1.09]
stale first edit A / C; stale in final diff	15/45 / 14/45, $p = 1.0$; 9/45 / 15/45 (exploratory; Fisher $p = 0.23$)
list cost per session A / C	$0.278 / $0.253
Opus 5.5 on the same 15 tasks	C/A tokens 0.81 [0.71, 0.92], cost 0.85 [0.78, 0.91]; pass 45/45 / 44/45
Later tool version C2, Opus 5.5 (55 sessions; 8 post-cutoff tasks; A, C, H from the new-task study)	
coverage C / C2; pass rate C2 / A / C	2/8 / 4/8; 100% (24/24) / 100% / 95.8%; non-inferior; controls 9/9
C2/C (pre-committed primary)	1.03 [0.88, 1.22] / 1.03 [0.89, 1.21] / 1.01 [0.90, 1.12]
C2/A; C2/H	0.82 [0.68, 1.01] / 0.86 [0.73, 1.04] / 0.78 [0.69, 0.87]; 0.95 [0.80, 1.09] / 0.91 [0.83, 0.998] / 0.91 [0.83, 0.98]
C2/C tokens: newly covered (pl01, mi01) / both (pl02, st01) / neither (4)	0.84 / 1.31 / 1.00 [0.84, 1.26] (descriptive)
drift (11 tasks): concurrent A / A, C / C; C2 / concurrent C	1.04 [0.86, 1.23], 1.11 [0.91, 1.33]; 0.83 [0.68, 1.01] (tokens)
stale first edit C2 / C / A	0/24 / 0/24 / 0/24
Case study, Opus 5.5 on a real repository (20 sessions; A, C; 2 tasks, 5 runs each; descriptive)	
pass A / C: pre-committed / corrected test	cs01 1/5 / 3/5, corrected 5/5 / 5/5 (post hoc); cs02 5/5 / 5/5
C/A per task, run-level bootstrap	cs01 0.74 [0.58, 1.20] / 0.86 [0.68, 1.09] / 0.79 [0.74, 1.00]; cs02 0.98 [0.59, 1.23] / 0.85 [0.68, 1.05] / 0.80 [0.53, 1.04]
list cost per session A / C	$0.515 / $0.432 (totals $5.15 / $4.32); stale first edit 0/10 / 0/10

Table 14: Follow-up studies (pre-committed in addendum sections 11, 12 and 13; section 14 was registered but not run, because the Claude CLI login had expired). Ratios are geometric means over tasks of per-task median ratios with task-cluster bootstrap CIs, except the case study (per task, over runs). In the R study a concurrent A, and in the C2 study a concurrent A and C, ran one repetition per task. The R study's C/R result did not replicate in the concurrent study.

quantity	value
A vs C, all 35 tasks (210 sessions); primary scope: 31 treatment tasks, 93 runs per arm	
pass rate A / C	19.4% (18/93) / 24.7% (23/93); Δ +5.4 [−4.3, +14.0] points, sign-flip p = 0.41 (10 tasks differ); McNemar on task majorities 4 vs 2, p = 0.69
C/A tokens / turns / wall time	1.08 [0.87, 1.32] / 1.01 [0.88, 1.14] / 0.92 [0.78, 1.08]
median per session A / C	tokens 67.3k / 73.0k; model calls 16 / 17; wall time 136 s / 126 s
stale first edit A / C	39.4% (26/66) / 32.4% (24/74), sign-flip p = 0.21 (13 of 28 tasks differ), Fisher p = 0.48; runs without a target-file edit 22 / 16
stale in final diff A / C	28.0% (26/93) / 18.3% (17/93) (descriptive)
covered (6) / uncovered (25) tasks	pass 6/18 / 6/18 and 12/75 / 17/75 (Δ +6.7 [0.0, +13.3], p = 0.13); tokens 1.37 [0.89, 2.27] and 1.01 [0.82, 1.26]; stale first edit 7/12 / 3/12 and 19/54 / 21/62
25 tasks post-cutoff for every model	pass 13.3% / 18.7%, Δ +5.3 [−1.3, +13.3], p = 0.31; tokens 1.08 [0.85, 1.36]
without pd01	Δ +4.4 [−5.6, +13.3], p = 0.53; tokens 1.09 [0.88, 1.34]
controls pl90, pl91 / other (oai04, oai05)	6/6 / 3/6 (descriptive; tokens 0.66) / 1/6 / 0/6
exits A / C (all 105 sessions each)	submitted 70 / 78; format error 28 / 25; context overflow 6 / 2; call limit 1 / 0
A, C and H on the 17 original post-cutoff tasks (51 runs per arm; H added by amendment, not randomised with A and C)	
pass rate A / C / H	3.9% (2/51) / 9.8% (5/51) / 3.9% (2/51); floor rule fired, no pass-rate claim
C/A; H/A; C/H tokens	1.12 [0.84, 1.49]; 0.79 [0.55, 1.07]; 1.42 [1.04, 2.02]
C/A; H/A; C/H turns	1.06 [0.89, 1.25]; 0.84 [0.69, 1.00]; 1.26 [1.04, 1.55]
stale first edit A / C / H	29.7% (11/37) / 34.1% (14/41) / 27.3% (9/33); all sign-flip p = 1.0
exits A / C / H	submitted 32 / 42 / 32; format error 14 / 7 / 16; context overflow 4 / 2 / 3; call limit 1 / 0 / 0

Table 15: Open-weight agent (Qwen3-Coder-30B-A3B in mini-swe-agent; pre-committed in addendum section 10, H in amendment 10a). Ratios are geometric means over tasks of per-task median ratios with task-cluster bootstrap CIs; p is the exact sign-flip test unless stated. Wall time of H is not comparable (H partly shared the server with late A and C sessions).

		recall: share or count of the documented changes reported						precision (56 reports)	
tool	reports	all 154 % [95% CI]	no behav. (122)	removal (42)	rename (22)	sign. (54)	behav. (32)	accurate	breaking
0.4.1 (evaluated)	1,170	48.7 [40.9, 56.5]	61.5%	41	12	22	0	100% [93.6, 100]	83.9%
dev. version (C2)	1,270	57.1 [49.3, 64.7]	72.1%	42	18	24	0	100%	83.9%
`griffe check` 2.4.1	1,722	55.2 [47.3, 62.8]	69.7%	42	18	24	0	89.3% [78.5, 95.0]	85.7%
AexPy 0.4.3	5,394	53.2 [45.4, 60.9]	67.2%	39	14	26	0	87.5% [76.4, 93.8]	44.6%
union of the four		61.7 [53.8, 69.0]	77.9%				0		

What reaches the block (0.4.1 / C2; a project that only imports the package): five-line block (benchmark setting) 8.4% [5.0, 13.9] / 10.4% of the 154 items; of the items the diff found, 17.3% [10.4, 27.4] / 18.2% reach a five-line and 36.0% / 35.2% a ten-line block. The 0.4.1 five-line block holds 11 of 42 removals and 2 of 54 signature changes; the union of the frozen benchmark blocks names 15 of 154 (9.7%); 35 of 35 block lines are accurate.

Table 16: Detector recall against 154 documented breaking or behavioural changes (seven libraries, cutoff-era to pinned release) and precision on random samples of 8 reports per library and tool (AI-judged, no human review). Wilson 95% intervals, items pooled across libraries. Of the 3 dependency switches and the one default change, 0.4.1 found none, C2 all four, `griffe check` the default change and AexPy the switches (indirectly). "Breaking": accurate and breaking for code on a public path; the rest of 0.4.1's and C2's reports are new deprecations, while AexPy also reports private API.

per repository, cutoff 2026-06-30	G: random (100)	AI-stack (100)	earlier 47 (top-starred)
stars, median [IQR]	695 [307, 1,577]	769 [367, 2,567]	top of the ranking
direct dependencies, median [IQR]	19 [9, 36.5]	34.5 [19, 53]	36 [23.5, 69.5]
$\geq$1 pinned direct dependency released after the cutoff	**53%** [43, 63]	**93%** [88, 97]	77% [63, 89]
share of pinned direct dependencies locked after the cutoff, median	5.9%	38%	15%
imports a pinned package with a breaking change	**44%** [34, 54]	**91%** [85, 96]	64% [50, 78]
matcher finds a changed-API use (pinned package)	**12%** [6, 19]	**40%** [30, 50]	21% [10, 34]
matched changed APIs / changes (sum)	124 / 298	155 / 401	53 / 128
imported-scope block non-empty	47% [37, 57]	96% [92, 99]	66% [52, 80]
block tokens when non-empty, median [IQR]	704 [435, 1,134]	1,025 [622, 1,728]	764 [574, 1,329]
block tokens when non-empty, mean / max	1,014 / 3,814	1,320 / 5,580	1,306 / 11,100
packages / change lines in a non-empty block, median	2 / 10	5 / 20	4 / 14
blocks over 1,000 / 2,000 tokens	15 / 5	50 / 18	11 / 3
used-scope (default) block non-empty	13%	42%	21%
scan time per repository, median	9.6 s	62.5 s	25.8 s

Table 17: Exposure in the random two-stratum sample and, under the same definitions, in the earlier convenience sample (Table 10). Bold: the three exposure rates. Brackets after shares: owner-cluster bootstrap 95% intervals. AI$-$G differences: $+40$ [29, 51], $+47$ [36, 58] and $+28$ [16, 39] points (approximate, as 19 repositories are in both strata). "Pinned" excludes the tool's fallback to the latest release on PyPI.

second audit (60 matches)	value [95% CI]
precision (true / audited)	**48/60 = 80%**, Wilson 68–88, repository-cluster 61–93
false positives	11 wrong object (5 top-level `anyio` names taken for deprecated aliases; 6 calls on the project's own object or another class), 1 not breaking
without the 18-match repository; repository-weighted	30/42 = 71% (Wilson 56–83); 77%
true positives by use	42 working form, 1 version-gated, 5 old form (3 legacy `httpx` objects, 2 deprecated functions)
runtime: breaks	**0/60**, Wilson 0–6.0
runtime: warns only	2/60 (3.3%), Wilson 0.9–11.4
runtime: runs	46/60 (77%), repository-cluster 57–91
basis of the runtime class	14 snippet, 34 static
repositories with a true positive / break / old-form use	21 / 0 / 5 of 24
tool's old-form labels correct	4 of 16

Table 18: AI-judged audit of 60 random changed-API matches in the random sample (Claude Opus 5.5; no human review).

		f_{upper} (imports a changed package)		f_{strict} (uses a changed API)	
analysis	contexts	all	Python-editing	all	Python-editing
primary	1,308	**7.0** [2.4, 16.2]	**17.7** [11.2, 21.9]	**0.8** [0.1, 2.2]	**2.1** [0.3, 3.4]
breaking changes only	1,308	5.9 [2.3, 13.4]	15.0 [10.9, 18.1]	–	–
S1 without the tool's own repository	1,167	4.7 [2.0, 12.9]	14.6 [10.1, 18.6]	0.6	1.9
S2 without scratch-only Python edits	1,308	6.0 [1.9, 14.8]	18.5 [10.2, 22.9]	0.5	1.4
S1 and S2	1,167	3.7 [1.5, 10.8]	14.7 [8.8, 19.5]	0.2	0.7
S3 unchecked packages counted as changed	1,308	8.6 [2.9, 20.2]	21.8 [13.5, 28.1]	0.8	2.1
S4 Opus 5.5's cutoff for every context	1,308	6.9 [2.4, 16.0]	17.5 [10.9, 21.8]	0.8	2.1
S5 Opus 5.5 contexts only	1,120	7.6 [2.6, 18.2]	17.7 [10.8, 22.5]	0.7	1.7
S7 complete cases (every header recovered)	262[a]	–	24.8 [16.4, 31.4]	–	–
S7 missing headers imputed	1,308	11.3 [5.1, 25.6]	28.7 [23.5, 33.1][b]	–	–
S6 missing headers counted as exposed	1,308	24.2 [12.2, 54.6]	61.6 [49.6, 85.6]	–	–

Table 19: Share of agent contexts (%, project-clustered bootstrap 95% CI) in the author's Claude Code sessions, May to October 2026. *All*: every included context; *Python-editing*: those that edited Python (513 in the primary analysis, 377 in S1, 426 in S2, 293 in S1 and S2, 481 in S5). A package is changed if the tool's static diff finds a public API change between the release current at the model's cutoff and the version in use. Intervals of the f_{strict} sensitivity rows are in the data folder (`analysis/f_sessions.json`). S2 does not count a context as Python-editing if all its Python edits are in temporary folders; S7 imputes each project category's complete-case rate; S6 and S7 were added after seeing the share of missing headers. [a]Python-editing contexts. [b]Imputed rates held fixed in the bootstrap, so this interval is too narrow.

contexts	n	med.	mean t	c (\$)	$f^*_{u=0}$
benchmark	17[a]	–	11.3[c]	0.0072	8.3%
main sessions	27	1	131.8[b]	0.0242	27.7%
subagent runs	1,281	30	43.4	0.0118	13.5%
all (f's base)	1,308	29	45.3	0.0120	13.8%
Python-editing	513	63	81.6	0.0171	19.6%
write only	–	–	0	0.0056	6.5%

Table 20: Cost of the 704-token block per context in the transcripts behind f (post hoc): model calls per context (median, mean t), $c = b\,(t\,p_r + p_w)$ at the mean, and break-even f^* at $u = 0$ with the main study's s. Write only: one cache write, no reads. With K measured in the same contexts (all: \$2.96; Python-editing: \$5.97), c/K, the u from which the block pays at every f, is 0.40% and 0.29%. [a]Main-study tasks (benchmark session). [b]One session has 3,341 calls. [c]Model calls; the CLI reports 16.6 turns, which count tool round trips.